

Smart Campus Parking and Traffic Management System

CSA0925-Programming in Java

S. Dharanish(192472152)

P. Jeevan Kumar(192472082)

Akileshwaran A(192511354)

PseudoCode:

```
package com.campus.parking.dao;

import com.campus.parking.database.DBConnection;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.time.Duration;
import java.time.LocalDateTime;

public class SessionDAO {

    public int createSession(int vehicleId, int slotId) {

        String sql = "INSERT INTO parking_sessions " +
            "(vehicle_id, slot_id, entry_time, session_status) " +
            "VALUES (?, ?, NOW(), 'Active')";

        try (
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(
                sql,
                Statement.RETURN_GENERATED_KEYS
            )
        ) {

            ps.setInt(1, vehicleId);
            ps.setInt(2, slotId);
```

```

        ps.executeUpdate();

        try (ResultSet rs = ps.getGeneratedKeys()) {
            if (rs.next()) {
                return rs.getInt(1);
            }
        }

    } catch (SQLException e) {
        System.out.println("Session creation error: " + e.getMessage());
    }

    return -1;
}

public boolean updateSlotStatus(int slotId, String status) {

    String sql = "UPDATE parking_slots " +
        "SET slot_status = ? " +
        "WHERE slot_id = ?";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setString(1, status);
        ps.setInt(2, slotId);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.out.println("Slot update error: " + e.getMessage());
        return false;
    }
}

public int findSlotId(String slotNumber) {

```

```

String sql = "SELECT slot_id FROM parking_slots " +
    "WHERE slot_number = ?";

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql)
) {

    ps.setString(1, slotNumber);

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            return rs.getInt("slot_id");
        }
    }

} catch (SQLException e) {
    System.out.println("Slot search error: " + e.getMessage());
}

return -1;
}

```

```

public int findVehicleId(String vehicleNumber) {

    String sql = "SELECT vehicle_id FROM vehicles " +
        "WHERE vehicle_number = ?";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setString(1, vehicleNumber);

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getInt("vehicle_id");
            }
        }
    }
}

```

```

    } catch (SQLException e) {
        System.out.println("Vehicle search error: " + e.getMessage());
    }

    return -1;
}

public boolean createVehicle(
    int userId,
    String vehicleNumber,
    String vehicleType
) {

    String sql = "INSERT INTO vehicles " +
        "(user_id, vehicle_number, vehicle_type) " +
        "VALUES (?, ?, ?)";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setInt(1, userId);
        ps.setString(2, vehicleNumber);
        ps.setString(3, vehicleType);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.out.println("Vehicle creation error: " + e.getMessage());
        return false;
    }
}

public boolean vehicleExists(String vehicleNumber) {

    String sql = "SELECT COUNT(*) FROM vehicles " +
        "WHERE vehicle_number = ?";

```

```

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql)
) {

    ps.setString(1, vehicleNumber);

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            return rs.getInt(1) > 0;
        }
    }

} catch (SQLException e) {
    System.out.println("Vehicle check error: " + e.getMessage());
}

return false;
}

public int getActiveSessionCount() {

    String sql = "SELECT COUNT(*) FROM parking_sessions " +
        "WHERE session_status = 'Active'";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()
    ) {

        if (rs.next()) {
            return rs.getInt(1);
        }

    } catch (SQLException e) {
        System.out.println("Active session error: " + e.getMessage());
    }

    return 0;
}

```

```
}
```

```
public double calculateFee(long minutes) {
```

```
    if (minutes <= 0) {  
        return 2.00;  
    }
```

```
    long hours = (minutes + 59) / 60;
```

```
    return 2.00 + (hours * 1.50);  
}
```

```
public double calculateFee(  
    LocalDateTime entryTime,  
    LocalDateTime exitTime  
) {
```

```
    if (entryTime == null || exitTime == null) {  
        return 2.00;  
    }
```

```
    if (exitTime.isBefore(entryTime)) {  
        return 2.00;  
    }
```

```
    long minutes = Duration  
        .between(entryTime, exitTime)  
        .toMinutes();
```

```
    return calculateFee(minutes);  
}
```

```
public boolean isSlotAvailable(int slotId) {
```

```
    String sql = "SELECT slot_status FROM parking_slots " +  
        "WHERE slot_id = ?";
```

```
    try (  
        Connection con = DBConnection.getConnection();
```

```

        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setInt(1, slotId);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                String status =
                    rs.getString("slot_status");

                return "Available".equalsIgnoreCase(status);
            }
        }

    } catch (SQLException e) {
        System.out.println(
            "Slot availability error: " + e.getMessage()
        );
    }

    return false;
}

public boolean hasActiveSessionForVehicle(int vehicleId) {

    String sql = "SELECT COUNT(*) FROM parking_sessions " +
        "WHERE vehicle_id = ? " +
        "AND session_status = 'Active'";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setInt(1, vehicleId);

        try (ResultSet rs = ps.executeQuery()) {

```

```

        if (rs.next()) {
            return rs.getInt(1) > 0;
        }
    }

} catch (SQLException e) {
    System.out.println(
        "Active vehicle session error: "
        + e.getMessage()
    );
}

return false;
}

public boolean checkInVehicle(
    int vehicleId,
    int slotId
) {

    Connection con = null;

    try {

        con = DBConnection.getConnection();

        con.setAutoCommit(false);

        String checkSlot =
            "SELECT slot_status FROM parking_slots " +
            "WHERE slot_id = ? FOR UPDATE";

        try (PreparedStatement ps =
            con.prepareStatement(checkSlot)) {

            ps.setInt(1, slotId);

            try (ResultSet rs = ps.executeQuery()) {

                if (!rs.next()) {

```



```

        con.rollback();
        return false;
    }

    String status =
        rs.getString("slot_status");

    if (!"Available".equalsIgnoreCase(status)) {
        con.rollback();
        return false;
    }
}

String sessionSql =
    "INSERT INTO parking_sessions " +
        "(vehicle_id, slot_id, entry_time, session_status) " +
        "VALUES (?, ?, NOW(), 'Active')";

try (PreparedStatement ps =
    con.prepareStatement(sessionSql)) {

    ps.setInt(1, vehicleId);
    ps.setInt(2, slotId);

    ps.executeUpdate();
}

String slotSql =
    "UPDATE parking_slots " +
        "SET slot_status = 'Occupied' " +
        "WHERE slot_id = ?";

try (PreparedStatement ps =
    con.prepareStatement(slotSql)) {

    ps.setInt(1, slotId);

    ps.executeUpdate();
}

```

```

        con.commit();

        return true;

    } catch (SQLException e) {

        try {
            if (con != null) {
                con.rollback();
            }
        } catch (SQLException ignored) {
        }

        System.out.println(
            "Check-in error: " + e.getMessage()
        );

        return false;

    } finally {

        try {

            if (con != null) {
                con.setAutoCommit(true);
                con.close();
            }

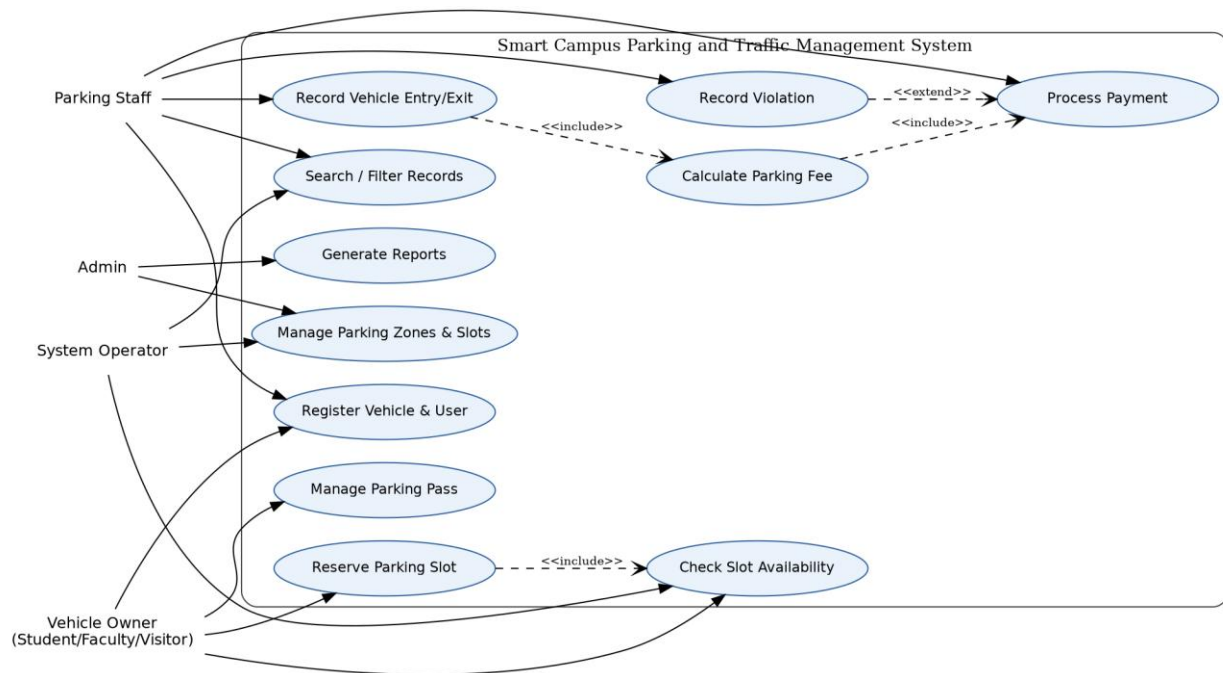
        } catch (SQLException ignored) {
        }

    }

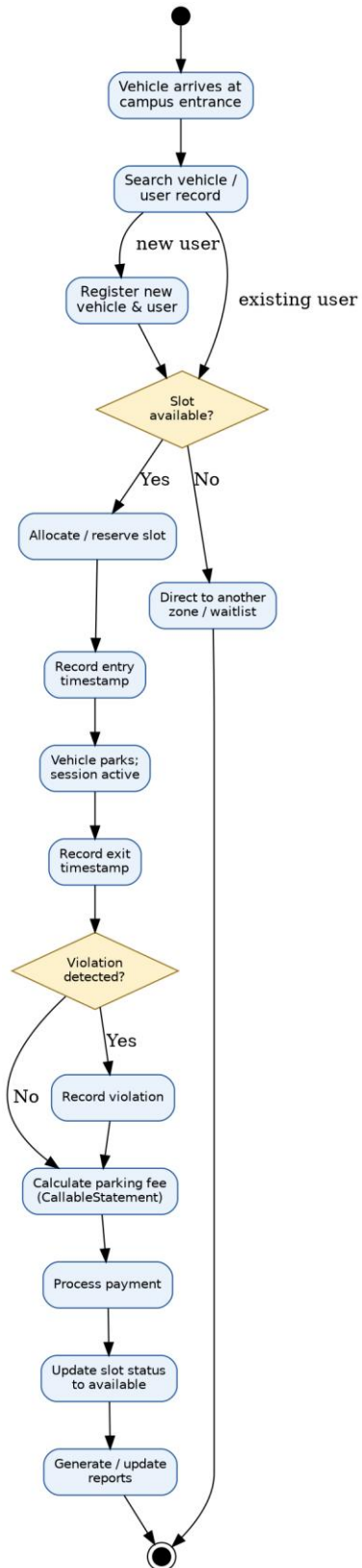
}

```

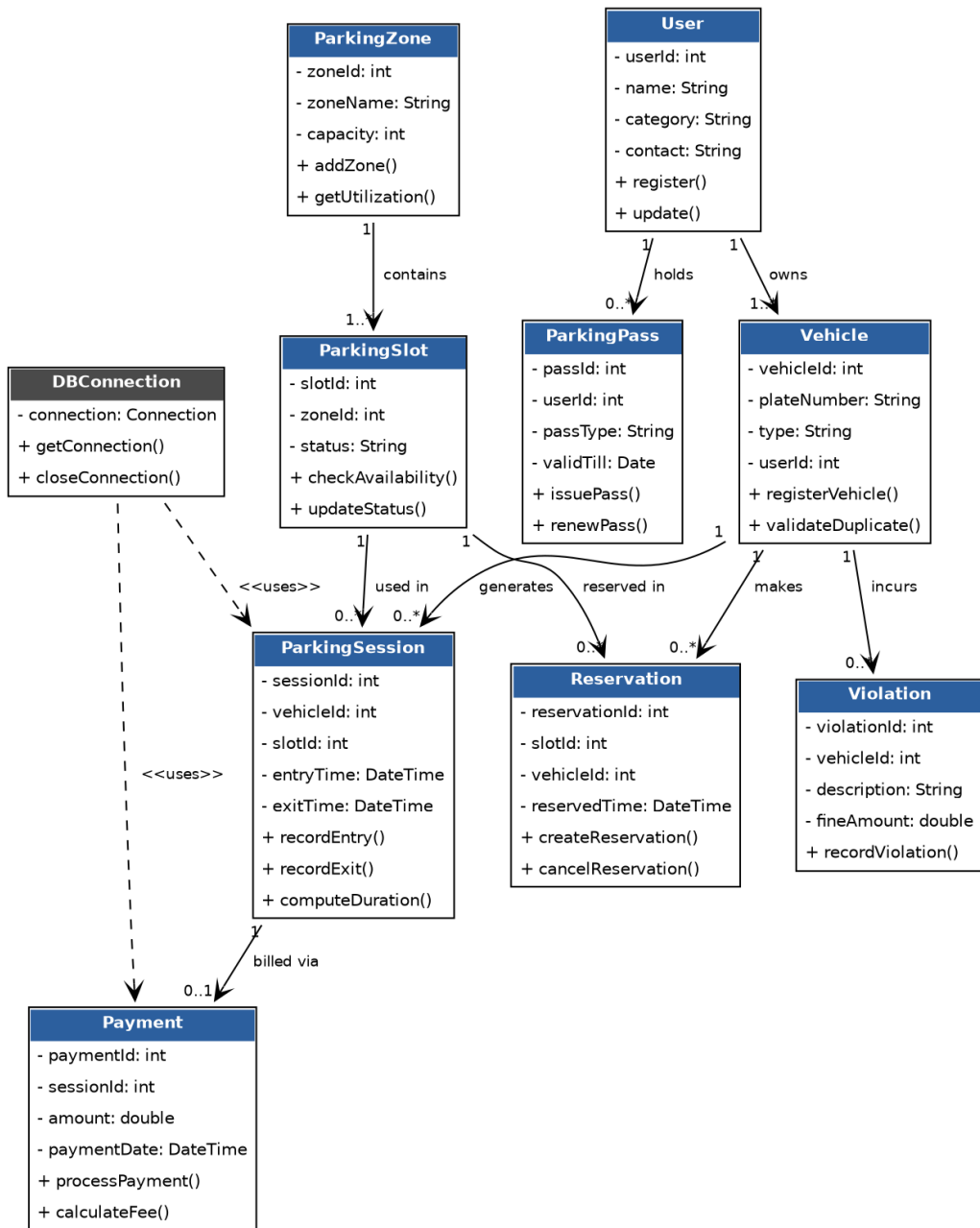
Case diagram



Activity Diagram



Class Diagram:



```
package com.campus.parking.dao;

import java.time.LocalDateTime;

public class SessionTest {

    public static void main(String[] args) {

        SessionDAO dao = new SessionDAO();

        System.out.println("===== PARKING SESSION TEST =====");

        int activeSessions =
            dao.getActiveSessionCount();

        System.out.println(
            "Active Parking Sessions: "
                + activeSessions
        );

        LocalDateTime entry =
            LocalDateTime.now().minusMinutes(90);

        LocalDateTime exit =
            LocalDateTime.now();

        double fee =
            dao.calculateFee(
                entry,
                exit
            );

        System.out.printf(
            "Sample 90 Minute Fee: ₹%.2f%n",
            fee
        );
    }
}
```

```

        System.out.println(
            "=====
        );
    }
}

package com.campus.parking.dao;

import com.campus.parking.database.DBConnection;
import com.campus.parking.model.ParkingSlot;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class SlotDAO {

    public List<ParkingSlot> getAllSlots() {

        List<ParkingSlot> slots = new ArrayList<>();

        String sql = ""
            SELECT slot_id, zone_id, slot_number, slot_status
            FROM parking_slots
            ORDER BY zone_id, slot_id
            "";

        try (
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()
        ) {

            while (rs.next()) {

                ParkingSlot slot = new ParkingSlot();

                slot.setSlotId(rs.getInt("slot_id"));
                slot.setZoneId(rs.getInt("zone_id"));
                slot.setSlotNumber(rs.getString("slot_number"));
                slot.setSlotStatus(rs.getString("slot_status"));
            }
        }
    }
}

```

```

        slots.add(slot);
    }

} catch (SQLException e) {
    System.out.println("Error loading parking slots: " + e.getMessage());
}

return slots;
}

public ParkingSlot getSlotById(int slotId) {

    String sql = ""
        SELECT slot_id, zone_id, slot_number, slot_status
        FROM parking_slots
        WHERE slot_id = ?
        """;

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setInt(1, slotId);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                ParkingSlot slot = new ParkingSlot();

                slot.setSlotId(rs.getInt("slot_id"));
                slot.setZoneId(rs.getInt("zone_id"));
                slot.setSlotNumber(rs.getString("slot_number"));
                slot.setSlotStatus(rs.getString("slot_status"));

                return slot;
            }
        }
    }
}

```



```

    } catch (SQLException e) {
        System.out.println("Error finding slot: " + e.getMessage());
    }

    return null;
}

public ParkingSlot getSlotByNumber(String slotNumber) {

    String sql = ""
        SELECT slot_id, zone_id, slot_number, slot_status
        FROM parking_slots
        WHERE slot_number = ?
        """;

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setString(1, slotNumber);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                ParkingSlot slot = new ParkingSlot();

                slot.setSlotId(rs.getInt("slot_id"));
                slot.setZoneId(rs.getInt("zone_id"));
                slot.setSlotNumber(rs.getString("slot_number"));
                slot.setSlotStatus(rs.getString("slot_status"));

                return slot;
            }
        }
    }

    } catch (SQLException e) {
        System.out.println("Error finding slot by number: " + e.getMessage());
    }
}

```

```

    }

    return null;
}

public ParkingSlot getFirstAvailableSlot() {

    String sql = ""
        SELECT slot_id, zone_id, slot_number, slot_status
        FROM parking_slots
        WHERE slot_status = 'Available'
        ORDER BY zone_id, slot_id
        LIMIT 1
        "";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()
    ) {

        if (rs.next()) {

            ParkingSlot slot = new ParkingSlot();

            slot.setSlotId(rs.getInt("slot_id"));
            slot.setZoneId(rs.getInt("zone_id"));
            slot.setSlotNumber(rs.getString("slot_number"));
            slot.setSlotStatus(rs.getString("slot_status"));

            return slot;
        }

    } catch (SQLException e) {
        System.out.println("Error finding available slot: " + e.getMessage());
    }

    return null;
}

```

```

public boolean updateSlotStatus(int slotId, String status) {

    String sql = ""
        UPDATE parking_slots
        SET slot_status = ?
        WHERE slot_id = ?
        "";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setString(1, status);
        ps.setInt(2, slotId);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.out.println("Error updating slot: " + e.getMessage());
        return false;
    }
}

```

```

public boolean occupySlot(int slotId) {

    String sql = ""
        UPDATE parking_slots
        SET slot_status = 'Occupied'
        WHERE slot_id = ?
        AND slot_status = 'Available'
        "";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setInt(1, slotId);
    }
}

```

```

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.out.println("Error occupying slot: " + e.getMessage());
        return false;
    }
}

```

```

public boolean releaseSlot(int slotId) {

```

```

    String sql = ""
        UPDATE parking_slots
        SET slot_status = 'Available'
        WHERE slot_id = ?
        """;

```

```

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        ps.setInt(1, slotId);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.out.println("Error releasing slot: " + e.getMessage());
        return false;
    }
}

```

```

public List<ParkingSlot> getAvailableSlots() {

```

```

    List<ParkingSlot> slots = new ArrayList<>();

```

```

    String sql = ""
        SELECT slot_id, zone_id, slot_number, slot_status
        FROM parking_slots
        WHERE slot_status = 'Available'
        ORDER BY zone_id, slot_id

```

```

        """;

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()
) {

    while (rs.next()) {

        ParkingSlot slot = new ParkingSlot();

        slot.setSlotId(rs.getInt("slot_id"));
        slot.setZoneId(rs.getInt("zone_id"));
        slot.setSlotNumber(rs.getString("slot_number"));
        slot.setSlotStatus(rs.getString("slot_status"));

        slots.add(slot);
    }

} catch (SQLException e) {
    System.out.println("Error loading available slots: " + e.getMessage());
}

return slots;
}

public int getAvailableCount() {

    String sql = ""
        SELECT COUNT(*)
        FROM parking_slots
        WHERE slot_status = 'Available'
        """;

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps = con.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()
) {

```

```

        if (rs.next()) {
            return rs.getInt(1);
        }

    } catch (SQLException e) {
        System.out.println("Error counting available slots: " + e.getMessage());
    }

    return 0;
}

public int getOccupiedCount() {

    String sql = ""
        SELECT COUNT(*)
        FROM parking_slots
        WHERE slot_status = 'Occupied'
        "";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()
    ) {

        if (rs.next()) {
            return rs.getInt(1);
        }

    } catch (SQLException e) {
        System.out.println("Error counting occupied slots: " + e.getMessage());
    }

    return 0;
}

public int getReservedCount() {

    String sql = ""

```

```

        SELECT COUNT(*)
        FROM parking_slots
        WHERE slot_status = 'Reserved'
        """;

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()
    ) {

        if (rs.next()) {
            return rs.getInt(1);
        }

    } catch (SQLException e) {
        System.out.println("Error counting reserved slots: " + e.getMessage());
    }

    return 0;
}
}

package com.campus.parking.dao;

import com.campus.parking.model.ParkingSlot;

import java.util.List;

public class SlotTest {

    public static void main(String[] args) {

        SlotDAO dao = new SlotDAO();

        List<ParkingSlot> slots =
            dao.getAllSlots();

        System.out.println(
            "===== CAMPUS PARKING SLOTS ====="
        );
    }
}

```

```

for (ParkingSlot slot : slots) {

    System.out.println(
        "Slot: "
            + slot.getSlotNumber()
            + " | Zone: "
            + slot.getZoneId()
            + " | Status: "
            + slot.getSlotStatus()
    );
}

System.out.println(
    "=====");

System.out.println(
    "Available: "
        + dao.getAvailableCount()
);

System.out.println(
    "Occupied: "
        + dao.getOccupiedCount()
);

System.out.println(
    "Reserved: "
        + dao.getReservedCount()
);
}
}

package com.campus.parking.dao;

import com.campus.parking.database.DBConnection;
import com.campus.parking.model.User;

import java.sql.*;
import java.util.ArrayList;

```



```
import java.util.List;

public class UserDao {

    public List<User> getAllUsers() {

        List<User> users = new ArrayList<>();

        String sql = ""
            SELECT user_id, name, email, phone, user_type
            FROM users
            ORDER BY user_id
            """;

        try (
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()
        ) {

            while (rs.next()) {

                User user = new User();

                user.setUserId(
                    rs.getInt("user_id")
                );

                user.setName(
                    rs.getString("name")
                );

                user.setEmail(
                    rs.getString("email")
                );

                user.setPhone(
                    rs.getString("phone")
                );
            }
        }
    }
}
```

```

        user.setUserType(
            rs.getString("user_type")
        );

        users.add(user);
    }

} catch (SQLException e) {

    System.out.println(
        "Error loading users: "
        + e.getMessage()
    );
}

return users;
}

public User getUserById(int userId) {

    String sql = ""
        SELECT user_id, name, email, phone, user_type
        FROM users
        WHERE user_id = ?
        "";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setInt(1, userId);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                User user = new User();

```

```

        user.setUserId(
            rs.getInt("user_id")
        );

        user.setName(
            rs.getString("name")
        );

        user.setEmail(
            rs.getString("email")
        );

        user.setPhone(
            rs.getString("phone")
        );

        user.setUserType(
            rs.getString("user_type")
        );

        return user;
    }
}

} catch (SQLException e) {

    System.out.println(
        "Error finding user: "
        + e.getMessage()
    );
}

return null;
}

public boolean addUser(User user) {

    String sql = ""
        INSERT INTO users
        (name, email, phone, user_type)

```

```

VALUES (?, ?, ?, ?)
""";

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps =
        con.prepareStatement(sql)
) {

    ps.setString(1, user.getName());
    ps.setString(2, user.getEmail());
    ps.setString(3, user.getPhone());
    ps.setString(4, user.getUserType());

    return ps.executeUpdate() > 0;

} catch (SQLException e) {

    System.out.println(
        "Error adding user: "
        + e.getMessage()
    );

    return false;
}
}

public boolean updateUser(User user) {

    String sql = ""
        UPDATE users
        SET name = ?,
            email = ?,
            phone = ?,
            user_type = ?
        WHERE user_id = ?
        """;

    try (
        Connection con = DBConnection.getConnection();

```

```

        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setString(1, user.getName());
        ps.setString(2, user.getEmail());
        ps.setString(3, user.getPhone());
        ps.setString(4, user.getUserType());
        ps.setInt(5, user.getUserId());

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {

        System.out.println(
            "Error updating user: "
                + e.getMessage()
        );

        return false;
    }
}

public boolean deleteUser(int userId) {

    String sql = ""
        DELETE FROM users
        WHERE user_id = ?
        """;

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setInt(1, userId);

        return ps.executeUpdate() > 0;
    }
}

```

```

    } catch (SQLException e) {

        System.out.println(
            "Error deleting user: "
                + e.getMessage()
        );

        return false;
    }
}

public List<User> searchUsers(String keyword) {

    List<User> users = new ArrayList<>();

    String sql = ""
        SELECT user_id, name, email, phone, user_type
        FROM users
        WHERE name LIKE ?
            OR email LIKE ?
            OR phone LIKE ?
        ORDER BY user_id
    """,

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        String search =
            "%" + keyword + "%";

        ps.setString(1, search);
        ps.setString(2, search);
        ps.setString(3, search);

        try (ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

```

```

        User user = new User();

        user.setUserId(
            rs.getInt("user_id")
        );

        user.setName(
            rs.getString("name")
        );

        user.setEmail(
            rs.getString("email")
        );

        user.setPhone(
            rs.getString("phone")
        );

        user.setUserType(
            rs.getString("user_type")
        );

        users.add(user);
    }
}

} catch (SQLException e) {

    System.out.println(
        "Error searching users: "
        + e.getMessage()
    );
}

return users;
}

public int getUserCount() {

```

```

String sql =
    "SELECT COUNT(*) FROM users";

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps =
        con.prepareStatement(sql);
    ResultSet rs =
        ps.executeQuery()
) {

    if (rs.next()) {
        return rs.getInt(1);
    }

} catch (SQLException e) {

    System.out.println(
        "Error counting users: "
        + e.getMessage()
    );
}

return 0;
}
}

package com.campus.parking.dao;

import com.campus.parking.model.User;

import java.util.List;

public class UserTest {

    public static void main(String[] args) {

        UserDAO dao = new UserDAO();

        List<User> users =
            dao.getAllUsers();

```



```

        System.out.println(
            "===== CAMPUS USER DIRECTORY ====="
        );

        for (User user : users) {

            System.out.println(
                "ID: " + user.getUserId()
                + " | Name: "
                + user.getName()
                + " | Email: "
                + user.getEmail()
                + " | Type: "
                + user.getUserType()
            );
        }

        System.out.println(
            "===== "
        );

        System.out.println(
            "Total Users: "
            + dao.getUserCount()
        );
    }
}

package com.campus.parking.dao;

import com.campus.parking.database.DBConnection;
import com.campus.parking.model.Violation;

import javax.swing.JOptionPane;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ViolationDAO {

```

```

public boolean addViolation(Violation violation) {

    String sql = ""
        INSERT INTO violations
        (user_id, vehicle_id, violation_type,
        fine_amount, violation_date, status)
        VALUES (?, ?, ?, ?, NOW(), ?)
        """;

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)
    ) {

        if (violation.getUserId() > 0) {
            ps.setInt(1, violation.getUserId());
        } else {
            ps.setNull(1, Types.INTEGER);
        }

        ps.setInt(2, violation.getVehicleId());

        ps.setString(
            3,
            violation.getViolationType()
        );

        ps.setDouble(
            4,
            violation.getFineAmount()
        );

        ps.setString(
            5,
            violation.getStatus()
        );

        int result = ps.executeUpdate();

        return result > 0;
    }
}

```

```

    } catch (SQLException e) {

        String message =
            "DATABASE ERROR\n\n"
            + e.getMessage();

        JOptionPane.showMessageDialog(
            null,
            message,
            "Violation Error",
            JOptionPane.ERROR_MESSAGE
        );

        System.out.println(
            "VIOLATION DATABASE ERROR: "
            + e.getMessage()
        );

        return false;
    }
}

```

```

public List<Violation> getAllViolations() {

```

```

    List<Violation> violations =
        new ArrayList<>();

```

```

    String sql = ""
        SELECT violation_id,
            user_id,
            vehicle_id,
            violation_type,
            fine_amount,
            violation_date,
            status
        FROM violations
        ORDER BY violation_id DESC
    """,

```

```

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps =
        con.prepareStatement(sql);
    ResultSet rs =
        ps.executeQuery()
) {

    while (rs.next()) {

        Violation violation =
            new Violation();

        violation.setViolationId(
            rs.getInt("violation_id")
        );

        violation.setUserId(
            rs.getInt("user_id")
        );

        violation.setVehicleId(
            rs.getInt("vehicle_id")
        );

        violation.setViolationType(
            rs.getString("violation_type")
        );

        violation.setFineAmount(
            rs.getDouble("fine_amount")
        );

        Timestamp timestamp =
            rs.getTimestamp(
                "violation_date"
            );

        if (timestamp != null) {

```

```

        violation.setViolationDate(
            timestamp.toLocalDateTime()
        );
    }

    violation.setStatus(
        rs.getString("status")
    );

    violations.add(violation);
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        null,
        "Error loading violations:\n\n"
        + e.getMessage(),
        "Database Error",
        JOptionPane.ERROR_MESSAGE
    );

    System.out.println(
        "ERROR LOADING VIOLATIONS: "
        + e.getMessage()
    );
}

return violations;
}

public boolean updateStatus(
    int violationId,
    String status
) {

    String sql = ""
        UPDATE violations
        SET status = ?
        WHERE violation_id = ?

```

```

        """,
    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setString(
            1,
            status
        );

        ps.setInt(
            2,
            violationId
        );

        return ps.executeUpdate() > 0;
    } catch (SQLException e) {

        JOptionPane.showMessageDialog(
            null,
            "Error updating violation:\n\n"
                + e.getMessage(),
            "Database Error",
            JOptionPane.ERROR_MESSAGE
        );

        return false;
    }
}

public boolean deleteViolation(
    int violationId
) {

    String sql = ""
        DELETE FROM violations

```

```

        WHERE violation_id = ?
        """,

try (
    Connection con = DBConnection.getConnection();
    PreparedStatement ps =
        con.prepareStatement(sql)
) {

    ps.setInt(
        1,
        violationId
    );

    return ps.executeUpdate() > 0;

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        null,
        "Error deleting violation:\n\n"
            + e.getMessage(),
        "Database Error",
        JOptionPane.ERROR_MESSAGE
    );

    return false;
}

}

public double getTotalFines() {

    String sql = ""
        SELECT COALESCE(
            SUM(fine_amount), 0
        )
        FROM violations
        """,

try (

```

```

        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql);
        ResultSet rs =
            ps.executeQuery()
    ) {

        if (rs.next()) {

            return rs.getDouble(1);
        }

    } catch (SQLException e) {

        System.out.println(
            "ERROR CALCULATING FINES: "
            + e.getMessage()
        );
    }

    return 0.0;
}

```

```

public int getViolationCount() {

    String sql =
        "SELECT COUNT(*) FROM violations";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql);
        ResultSet rs =
            ps.executeQuery()
    ) {

        if (rs.next()) {

            return rs.getInt(1);
        }

    }
}

```



```

    } catch (SQLException e) {

        System.out.println(
            "ERROR COUNTING VIOLATIONS: "
                + e.getMessage()
        );
    }

    return 0;
}

public int getUnpaidCount() {

    String sql = ""
        SELECT COUNT(*)
        FROM violations
        WHERE status = 'Unpaid'
        "";

    try (
        Connection con = DBConnection.getConnection();
        PreparedStatement ps =
            con.prepareStatement(sql);
        ResultSet rs =
            ps.executeQuery()
    ) {

        if (rs.next()) {

            return rs.getInt(1);
        }

    } catch (SQLException e) {

        System.out.println(
            "ERROR COUNTING UNPAID VIOLATIONS: "
                + e.getMessage()
        );
    }
}

```

```

        return 0;
    }
}

package com.campus.parking.dao;

public class ViolationTest {

    public static void main(String[] args) {

        ViolationDAO dao =
            new ViolationDAO();

        System.out.println(
            "===== VIOLATION REGISTRY ====="
        );

        System.out.println(
            "Total Violations: "
            + dao.getViolationCount()
        );

        System.out.println(
            "Total Fines: ₹"
            + String.format(
                "%.2f",
                dao.getTotalFines()
            )
        );

        System.out.println(
            "Unpaid Violations: "
            + dao.getUnpaidCount()
        );

        System.out.println(
            "===== "
        );
    }
}

```

```

package com.campus.parking.database;

import java.sql.Connection;
import java.sql.Statement;

public class DatabaseBootstrap {

    public static void initialize() {

        String[] queries = {

            """
            CREATE TABLE IF NOT EXISTS users (
                user_id INT PRIMARY KEY AUTO_INCREMENT,
                name VARCHAR(100) NOT NULL,
                email VARCHAR(100) NOT NULL UNIQUE,
                phone VARCHAR(15) NOT NULL,
                user_type ENUM(
                    'Student',
                    'Faculty',
                    'Staff',
                    'Visitor'
                ) NOT NULL
            )
            """,

            """
            CREATE TABLE IF NOT EXISTS vehicles (
                vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
                user_id INT NOT NULL,
                vehicle_number VARCHAR(20) NOT NULL UNIQUE,
                vehicle_type VARCHAR(30) NOT NULL,
                FOREIGN KEY (user_id)
                REFERENCES users(user_id)
            )
            """,

            """
            CREATE TABLE IF NOT EXISTS parking_zones (
                zone_id INT PRIMARY KEY AUTO_INCREMENT,

```

```

        zone_name VARCHAR(50) NOT NULL,
        location VARCHAR(100)
    )
    """,

    """

CREATE TABLE IF NOT EXISTS parking_slots (
    slot_id INT PRIMARY KEY AUTO_INCREMENT,
    zone_id INT NOT NULL,
    slot_number VARCHAR(20) NOT NULL,
    slot_status ENUM(
        'Available',
        'Occupied',
        'Reserved'
    ) DEFAULT 'Available',
    FOREIGN KEY (zone_id)
    REFERENCES parking_zones(zone_id)
)
    """,

    """

CREATE TABLE IF NOT EXISTS reservations (
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    vehicle_id INT NOT NULL,
    slot_id INT NOT NULL,
    reservation_date DATETIME
        DEFAULT CURRENT_TIMESTAMP,
    status ENUM(
        'Active',
        'Completed',
        'Cancelled'
    ) DEFAULT 'Active',
    FOREIGN KEY (user_id)
    REFERENCES users(user_id),
    FOREIGN KEY (vehicle_id)
    REFERENCES vehicles(vehicle_id),
    FOREIGN KEY (slot_id)
    REFERENCES parking_slots(slot_id)
)

```

""",

"""

```
CREATE TABLE IF NOT EXISTS parking_sessions (  
    session_id INT PRIMARY KEY AUTO_INCREMENT,  
    vehicle_id INT NOT NULL,  
    slot_id INT NOT NULL,  
    entry_time DATETIME  
        DEFAULT CURRENT_TIMESTAMP,  
    exit_time DATETIME NULL,  
    duration_minutes INT DEFAULT 0,  
    parking_amount DECIMAL(10,2)  
        DEFAULT 0.00,  
    session_status ENUM(  
        'Active',  
        'Completed'  
    ) DEFAULT 'Active',  
    FOREIGN KEY (vehicle_id)  
        REFERENCES vehicles(vehicle_id),  
    FOREIGN KEY (slot_id)  
        REFERENCES parking_slots(slot_id)  
)  
""",
```

"""

```
CREATE TABLE IF NOT EXISTS payments (  
    payment_id INT PRIMARY KEY AUTO_INCREMENT,  
    session_id INT NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    payment_method ENUM(  
        'Cash',  
        'UPI',  
        'Card'  
    ) NOT NULL,  
    payment_date DATETIME  
        DEFAULT CURRENT_TIMESTAMP,  
    payment_status ENUM(  
        'Paid',  
        'Pending'  
    ) DEFAULT 'Paid',
```

```

        FOREIGN KEY (session_id)
        REFERENCES parking_sessions(session_id)
    )
    """,

    """
CREATE TABLE IF NOT EXISTS violations (
    violation_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    vehicle_id INT NOT NULL,
    violation_type VARCHAR(100) NOT NULL,
    fine_amount DECIMAL(10,2)
        DEFAULT 0.00,
    violation_date DATETIME
        DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) NOT NULL,
    FOREIGN KEY (user_id)
    REFERENCES users(user_id),
    FOREIGN KEY (vehicle_id)
    REFERENCES vehicles(vehicle_id)
)
    """
};

try (
    Connection connection =
        DBConnection.getConnection();

    Statement statement =
        connection.createStatement()
) {

    for (String query : queries) {
        statement.executeUpdate(query);
    }

    System.out.println(
        "Database tables created successfully!"
    );
}

```

```

    } catch (Exception e) {

        System.out.println(
            "Database Bootstrap Error:"
        );

        System.out.println(
            e.getMessage()
        );
    }
}

package com.campus.parking.database;

import java.sql.Connection;

public class DatabaseTest {

    public static void main(String[] args) {

        try {

            Connection connection =
                DBConnection.getConnection();

            System.out.println(
                "=====
            );

            System.out.println(
                "DATABASE CONNECTED SUCCESSFULLY"
            );

            System.out.println(
                "Smart Campus Parking Sentinel"
            );

            System.out.println(
                "=====
            );

```

```

        connection.close();

    } catch (Exception e) {

        System.out.println(
            "DATABASE CONNECTION FAILED"
        );

        System.out.println(
            e.getMessage()
        );
    }
}

package com.campus.parking.database;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL =
        "jdbc:mysql://localhost:3306/smart_campus_parking_sentinel";

    private static final String USER = "root";

    private static final String PASSWORD = "123456";

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException(
                "MySQL JDBC Driver not found"
            );
        }
    }
}

```



```

    public static Connection getConnection()
        throws SQLException {

        return DriverManager.getConnection(
            URL,
            USER,
            PASSWORD
        );
    }
}

package com.campus.parking.model;

import java.time.LocalDateTime;

public class ParkingSession {

    private int sessionId;
    private int vehicleId;
    private int slotId;
    private LocalDateTime entryTime;
    private LocalDateTime exitTime;
    private int durationMinutes;
    private double parkingAmount;
    private String sessionStatus;

    public ParkingSession() {
    }

    public int getSessionId() {
        return sessionId;
    }

    public void setSessionId(int sessionId) {
        this.sessionId = sessionId;
    }

    public int getVehicleId() {
        return vehicleId;
    }
}

```

```
public void setVehicleId(int vehicleId) {  
    this.vehicleId = vehicleId;  
}
```

```
public int getSlotId() {  
    return slotId;  
}
```

```
public void setSlotId(int slotId) {  
    this.slotId = slotId;  
}
```

```
public LocalDateTime getEntryTime() {  
    return entryTime;  
}
```

```
public void setEntryTime(LocalDateTime entryTime) {  
    this.entryTime = entryTime;  
}
```

```
public LocalDateTime getExitTime() {  
    return exitTime;  
}
```

```
public void setExitTime(LocalDateTime exitTime) {  
    this.exitTime = exitTime;  
}
```

```
public int getDurationMinutes() {  
    return durationMinutes;  
}
```

```
public void setDurationMinutes(int durationMinutes) {  
    this.durationMinutes = durationMinutes;  
}
```

```
public double getParkingAmount() {  
    return parkingAmount;  
}
```

```
public void setParkingAmount(double parkingAmount) {
    this.parkingAmount = parkingAmount;
}

public String getSessionStatus() {
    return sessionStatus;
}

public void setSessionStatus(String sessionStatus) {
    this.sessionStatus = sessionStatus;
}
}

package com.campus.parking.model;

public class ParkingSlot {

    private int slotId;
    private int zoneId;
    private String slotNumber;
    private String slotStatus;

    public ParkingSlot() {
    }

    public ParkingSlot(
        int slotId,
        int zoneId,
        String slotNumber,
        String slotStatus
    ) {
        this.slotId = slotId;
        this.zoneId = zoneId;
        this.slotNumber = slotNumber;
        this.slotStatus = slotStatus;
    }

    public int getSlotId() {
        return slotId;
    }
}
```

```

    public void setSlotId(int slotId) {
        this.slotId = slotId;
    }

    public int getZoneId() {
        return zoneId;
    }

    public void setZoneId(int zoneId) {
        this.zoneId = zoneId;
    }

    public String getSlotNumber() {
        return slotNumber;
    }

    public void setSlotNumber(String slotNumber) {
        this.slotNumber = slotNumber;
    }

    public String getSlotStatus() {
        return slotStatus;
    }

    public void setSlotStatus(String slotStatus) {
        this.slotStatus = slotStatus;
    }

    @Override
    public String toString() {
        return slotNumber + " - " + slotStatus;
    }
}

package com.campus.parking.model;

public class User {

    private int userId;
    private String name;
    private String email;

```

```
private String phone;  
private String userType;
```

```
public User() {  
}
```

```
public User(  
    int userId,  
    String name,  
    String email,  
    String phone,  
    String userType  
) {  
    this.userId = userId;  
    this.name = name;  
    this.email = email;  
    this.phone = phone;  
    this.userType = userType;  
}
```

```
public User(  
    String name,  
    String email,  
    String phone,  
    String userType  
) {  
    this.name = name;  
    this.email = email;  
    this.phone = phone;  
    this.userType = userType;  
}
```

```
public int getUserId() {  
    return userId;  
}
```

```
public void setUserId(int userId) {  
    this.userId = userId;  
}
```

```

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPhone() {
    return phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}

public String getUserType() {
    return userType;
}

public void setUserType(String userType) {
    this.userType = userType;
}

@Override
public String toString() {
    return userId + " | " + name + " | " +
        email + " | " + phone + " | " +
        userType;
}
}

```

```
package com.campus.parking.model;
```

```
public class Vehicle {
```

```
    private int vehicleId;
```

```
    private int userId;
```

```
    private String vehicleNumber;
```

```
    private String vehicleType;
```

```
    public Vehicle() {
```

```
    }
```

```
    public Vehicle(
```

```
        int vehicleId,
```

```
        int userId,
```

```
        String vehicleNumber,
```

```
        String vehicleType
```

```
    ) {
```

```
        this.vehicleId = vehicleId;
```

```
        this.userId = userId;
```

```
        this.vehicleNumber = vehicleNumber;
```

```
        this.vehicleType = vehicleType;
```

```
    }
```

```
    public Vehicle(
```

```
        int userId,
```

```
        String vehicleNumber,
```

```
        String vehicleType
```

```
    ) {
```

```
        this.userId = userId;
```

```
        this.vehicleNumber = vehicleNumber;
```

```
        this.vehicleType = vehicleType;
```

```
    }
```

```
    public int getVehicleId() {
```

```
        return vehicleId;
```

```
    }
```

```
    public void setVehicleId(int vehicleId) {
```

```
        this.vehicleId = vehicleId;
```

```
}
```

```
public int getUserId() {  
    return userId;  
}
```

```
public void setUserId(int userId) {  
    this.userId = userId;  
}
```

```
public String getVehicleNumber() {  
    return vehicleNumber;  
}
```

```
public void setVehicleNumber(String vehicleNumber) {  
    this.vehicleNumber = vehicleNumber;  
}
```

```
public String getVehicleType() {  
    return vehicleType;  
}
```

```
public void setVehicleType(String vehicleType) {  
    this.vehicleType = vehicleType;  
}
```

```
@Override
```

```
public String toString() {  
    return vehicleId + " | " +  
        vehicleNumber + " | " +  
        vehicleType;  
}  
}
```

```
package com.campus.parking.model;
```

```
import java.time.LocalDateTime;
```

```
public class Violation {
```

```
    private int violationId;
```



```
private int userId;
private int vehicleId;
private String violationType;
private String description;
private double fineAmount;
private LocalDateTime violationDate;
private String status;

public Violation() {
}

public int getViolationId() {
    return violationId;
}

public void setViolationId(int violationId) {
    this.violationId = violationId;
}

public int getUserId() {
    return userId;
}

public void setUserId(int userId) {
    this.userId = userId;
}

public int getVehicleId() {
    return vehicleId;
}

public void setVehicleId(int vehicleId) {
    this.vehicleId = vehicleId;
}

public String getViolationType() {
    return violationType;
}

public void setViolationType(String violationType) {
```

```
        this.violationType = violationType;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getFineAmount() {
        return fineAmount;
    }

    public void setFineAmount(double fineAmount) {
        this.fineAmount = fineAmount;
    }

    public LocalDateTime getViolationDate() {
        return violationDate;
    }

    public void setViolationDate(LocalDateTime violationDate) {
        this.violationDate = violationDate;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}

package com.campus.parking.ui.components;

import javax.swing.*.*;
import java.awt.*.*;
```

```
public class KpiCard extends JPanel {

    private final JLabel titleLabel;
    private final JLabel valueLabel;
    private final JLabel subtitleLabel;

    private Color accentColor =
        new Color(56, 189, 248);

    public KpiCard(
        String title,
        String value,
        String subtitle
    ) {

        setLayout(
            new BorderLayout(
                8,
                4
            )
        );

        setOpaque(false);

        setBorder(
            BorderFactory.createEmptyBorder(
                18,
                20,
                18,
                20
            )
        );

        titleLabel =
            new JLabel(title);

        titleLabel.setFont(
            new Font(
                "Segoe UI",
                Font.PLAIN,
```

```
        13
    )
);

titleLabel.setForeground(
    new Color(148, 163, 184)
);

valueLabel =
    new JLabel(value);

valueLabel.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        30
    )
);

valueLabel.setForeground(
    Color.WHITE
);

subtitleLabel =
    new JLabel(subtitle);

subtitleLabel.setFont(
    new Font(
        "Segoe UI",
        Font.PLAIN,
        12
    )
);

subtitleLabel.setForeground(
    new Color(148, 163, 184)
);

JPanel textPanel =
    new JPanel();
```

```

textPanel.setOpaque(false);

textPanel.setLayout(
    new BoxLayout(
        textPanel,
        BoxLayout.Y_AXIS
    )
);

textPanel.add(titleLabel);
textPanel.add(valueLabel);
textPanel.add(subtitleLabel);

add(
    textPanel,
    BorderLayout.CENTER
);
}

public void setValue(
    String value
) {

    valueLabel.setText(value);
}

public void setAccentColor(
    Color color
) {

    accentColor = color;

    repaint();
}

@Override
protected void paintComponent(
    Graphics g
) {

```

```
Graphics2D g2 =
    (Graphics2D) g.create();

g2.setRenderingHint(
    RenderingHints.KEY_ANTIALIASING,
    RenderingHints.VALUE_ANTIALIAS_ON
);

g2.setColor(
    new Color(
        30,
        41,
        59
    )
);

g2.fillRoundRect(
    0,
    0,
    getWidth(),
    getHeight(),
    18,
    18
);

g2.setColor(accentColor);

g2.fillRoundRect(
    0,
    0,
    5,
    getHeight(),
    5,
    5
);

g2.setColor(
    new Color(
        51,
```

```

        65,
        85
    )
);

g2.drawRoundRect(
    0,
    0,
    getWidth() - 1,
    getHeight() - 1,
    18,
    18
);

g2.dispose();

super.paintComponent(g);
}
}

package com.campus.parking.ui.components;

import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

public class ModernButton extends JButton {

    private boolean hovered = false;
    private boolean active = false;

    private final Color normalColor = new Color(30, 41, 59);
    private final Color hoverColor = new Color(56, 189, 248);
    private final Color activeColor = new Color(99, 102, 241);
    private final Color textColor = new Color(226, 232, 240);

    public ModernButton(String text) {

        super(text);

```

```
setFont(new Font(
    "Segoe UI",
    Font.BOLD,
    14
));

setForeground(textColor);

setFocusPainted(false);
setBorderPainted(false);
setContentAreaFilled(false);
setOpaque(false);

setCursor(
    new Cursor(Cursor.HAND_CURSOR)
);

setBorder(
    BorderFactory.createEmptyBorder(
        12,
        18,
        12,
        18
    )
);

addMouseListener(
    new MouseAdapter() {

        @Override
        public void mouseEntered(
            MouseEvent e
        ) {
            hovered = true;
            repaint();
        }

        @Override
        public void mouseExited(
            MouseEvent e
```



```

        ) {
            hovered = false;
            repaint();
        }
    }
);
}

```

```

public void setActive(boolean active) {

```

```

    this.active = active;

    repaint();
}

```

```

public boolean isActive() {

```

```

    return active;
}

```

```

@Override

```

```

protected void paintComponent(
    Graphics g
) {

```

```

    Graphics2D g2 =
        (Graphics2D) g.create();

```

```

    g2.setRenderingHint(
        RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON
    );

```

```

    Color background;

```

```

    if (active) {

```

```

        background = activeColor;

```

```

    } else if (hovered) {

```

```
        background = hoverColor;

    } else {

        background = normalColor;
    }

    g2.setColor(background);

    g2.fillRoundRect(
        0,
        0,
        getWidth(),
        getHeight(),
        14,
        14
    );

    if (hovered || active) {

        g2.setColor(
            new Color(
                255,
                255,
                255,
                35
            )
        );

        g2.drawRoundRect(
            0,
            0,
            getWidth() - 1,
            getHeight() - 1,
            14,
            14
        );
    }
}
```

```

        g2.dispose();

        super.paintComponent(g);
    }
}

package com.campus.parking.ui.dialogs;

import javax.swing.*;
import java.awt.*;

public class CheckInDialog extends JDialog {

    private JTextField vehicleNumberField;
    private JTextField ownerNameField;
    private JTextField phoneField;
    private JComboBox<String> vehicleTypeCombo;
    private JComboBox<String> slotCombo;

    public CheckInDialog(Frame parent) {

        super(parent, "Vehicle Check-In", true);

        setSize(600, 600);
        setResizable(false);
        setLocationRelativeTo(parent);

        createUI();
    }

    private void createUI() {

        JPanel mainPanel = new JPanel();

        mainPanel.setLayout(
            new BoxLayout(
                mainPanel,
                BoxLayout.Y_AXIS
            )
        );
    }
}

```

```
mainPanel.setBackground(
    new Color(30, 43, 65)
);

mainPanel.setBorder(
    BorderFactory.createEmptyBorder(
        30, 35, 20, 35
    )
);

JLabel title =
    new JLabel("VEHICLE CHECK-IN");

title.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        28
    )
);

title.setForeground(Color.WHITE);

mainPanel.add(title);

mainPanel.add(
    Box.createVerticalStrut(8)
);

JLabel subtitle =
    new JLabel(
        "Register vehicle and assign parking slot"
    );

subtitle.setFont(
    new Font(
        "Segoe UI",
        Font.PLAIN,
        14
    )
);
```

```
);

subtitle.setForeground(
    new Color(160, 180, 215)
);

mainPanel.add(subtitle);

mainPanel.add(
    Box.createVerticalStrut(25)
);

JLabel vehicleLabel =
    createLabel("VEHICLE NUMBER");

mainPanel.add(vehicleLabel);

vehicleNumberField =
    new JTextField();

mainPanel.add(vehicleNumberField);

mainPanel.add(
    Box.createVerticalStrut(15)
);

JLabel ownerLabel =
    createLabel("OWNER NAME");

mainPanel.add(ownerLabel);

ownerNameField =
    new JTextField();

mainPanel.add(ownerNameField);

mainPanel.add(
    Box.createVerticalStrut(15)
);
```

```
JLabel phoneLabel =  
    createLabel("PHONE NUMBER");
```

```
mainPanel.add(phoneLabel);
```

```
phoneField =  
    new JTextField();
```

```
mainPanel.add(phoneField);
```

```
mainPanel.add(  
    Box.createVerticalStrut(15)  
);
```

```
JLabel typeLabel =  
    createLabel("VEHICLE TYPE");
```

```
mainPanel.add(typeLabel);
```

```
vehicleTypeCombo =  
    new JComboBox<(<br>        new String[]{  
            "Car",  
            "Bike",  
            "Scooter",  
            "Van",  
            "Other"  
        }  
    );
```

```
mainPanel.add(vehicleTypeCombo);
```

```
mainPanel.add(  
    Box.createVerticalStrut(15)  
);
```

```
JLabel slotLabel =  
    createLabel("PARKING SLOT");
```

```
mainPanel.add(slotLabel);
```

```

slotCombo =
    new JComboBox<>(
        new String[]{
            "A01",
            "A02",
            "A03",
            "A04",
            "A05",
            "B01",
            "B02",
            "B03",
            "B04",
            "B05",
            "C01",
            "C02",
            "C03",
            "C04",
            "C05",
            "D01",
            "D02",
            "D03",
            "D04",
            "D05"
        }
    );

mainPanel.add(slotCombo);

add(
    mainPanel,
    BorderLayout.CENTER
);

JPanel buttonPanel =
    new JPanel(
        new FlowLayout(
            FlowLayout.RIGHT
        )
    );

```

```

        buttonPanel.setBackground(
            new Color(13, 22, 42)
        );

        JButton cancelButton =
            new JButton("CANCEL");

        JButton checkInButton =
            new JButton("CHECK IN");

        cancelButton.addActionListener(
            e -> dispose()
        );

        checkInButton.addActionListener(
            e -> checkIn()
        );

        buttonPanel.add(cancelButton);
        buttonPanel.add(checkInButton);

        add(
            buttonPanel,
            BorderLayout.SOUTH
        );
    }

    private JLabel createLabel(String text) {

        JLabel label =
            new JLabel(text);

        label.setFont(
            new Font(
                "Segoe UI",
                Font.BOLD,
                13
            )
        );
    }

```



```
label.setForeground(  
    new Color(56, 189, 248)  
);  
  
label.setAlignmentX(  
    Component.LEFT_ALIGNMENT  
);  
  
return label;  
}
```

```
private void checkIn() {  
  
    String vehicle =  
        vehicleNumberField  
            .getText()  
            .trim();  
  
    String owner =  
        ownerNameField  
            .getText()  
            .trim();  
  
    String phone =  
        phoneField  
            .getText()  
            .trim();  
  
    String type =  
        String.valueOf(  
            vehicleTypeCombo  
                .getSelectedItem()  
        );  
  
    String slot =  
        String.valueOf(  
            slotCombo  
                .getSelectedItem()  
        );  
}
```

```
if (vehicle.isEmpty()) {

    JOptionPane.showMessageDialog(
        this,
        "Please enter vehicle number."
    );

    vehicleNumberField.requestFocus();

    return;
}

if (owner.isEmpty()) {

    JOptionPane.showMessageDialog(
        this,
        "Please enter owner name."
    );

    ownerNameField.requestFocus();

    return;
}

if (phone.isEmpty()) {

    JOptionPane.showMessageDialog(
        this,
        "Please enter phone number."
    );

    phoneField.requestFocus();

    return;
}

JOptionPane.showMessageDialog(
    this,
    "Vehicle Check-In Successful!\n\n"
```

```

        + "Vehicle: " + vehicle
        + "\nOwner: " + owner
        + "\nPhone: " + phone
        + "\nType: " + type
        + "\nSlot: " + slot,
        "Success",
        JOptionPane.INFORMATION_MESSAGE
    );

    dispose();
}
}

package com.campus.parking.ui.dialogs;

import com.campus.parking.database.DBConnection;
import com.campus.parking.dao.SessionDAO;

import javax.swing.*;
import java.awt.*;
import java.sql.*;
import java.time.Duration;
import java.time.LocalDateTime;

public class CheckOutDialog extends JDialog {

    private JTextField vehicleField;
    private JLabel vehicleInfo;
    private JLabel entryInfo;
    private JLabel durationInfo;
    private JLabel amountInfo;

    private JComboBox<String> paymentMethod;

    private final SessionDAO sessionDAO;

    private final Color ROOT = new Color(15, 23, 42);
    private final Color CARD = new Color(30, 41, 59);
    private final Color BORDER = new Color(51, 65, 85);
    private final Color TEXT = new Color(226, 232, 240);
    private final Color MUTED = new Color(148, 163, 184);

```

```
private final Color BLUE = new Color(56, 189, 248);
private final Color GREEN = new Color(16, 185, 129);
private final Color RED = new Color(239, 68, 68);
```

```
public CheckOutDialog(Frame parent) {

    super(parent, "Vehicle Exit & Payment", true);

    sessionDAO = new SessionDAO();

    setSize(560, 600);
    setLocationRelativeTo(parent);
    setResizable(false);

    createInterface();
}
```

```
private void createInterface() {

    JPanel main = new JPanel(
        new BorderLayout()
    );

    main.setBackground(ROOT);

    main.setBorder(
        BorderFactory.createEmptyBorder(
            25, 30, 25, 30
        )
    );

    JPanel header = new JPanel();

    header.setBackground(ROOT);

    header.setLayout(
        new BoxLayout(
            header,
            BoxLayout.Y_AXIS
        )
    );
```

```
);
```

```
JLabel title =  
    new JLabel("VEHICLE EXIT & PAYMENT");
```

```
title.setFont(  
    new Font(  
        "Segoe UI",  
        Font.BOLD,  
        25  
    )  
);
```

```
title.setForeground(TEXT);
```

```
JLabel subtitle =  
    new JLabel(  
        "Calculate parking duration and settle payment"  
    );
```

```
subtitle.setFont(  
    new Font(  
        "Segoe UI",  
        Font.PLAIN,  
        13  
    )  
);
```

```
subtitle.setForeground(MUTED);
```

```
header.add(title);
```

```
header.add(  
    Box.createVerticalStrut(5)  
);
```

```
header.add(subtitle);
```

```
main.add(  
    header,
```

```

        BorderLayout.NORTH
    );

    JPanel content =
        new JPanel(
            new GridBagLayout()
        );

    content.setBackground(CARD);

    content.setBorder(
        BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(
                BORDER
            ),
            BorderFactory.createEmptyBorder(
                20, 20, 20, 20
            )
        )
    );

    GridBagConstraints gbc =
        new GridBagConstraints();

    gbc.insets =
        new Insets(
            8, 8, 8, 8
        );

    gbc.fill =
        GridBagConstraints.HORIZONTAL;

    gbc.weightx = 1;

    int row = 0;

    JLabel vehicleLabel =
        createLabel(
            "Vehicle Number"
        );

```

```
vehicleField =  
    createTextField();  
  
JButton searchButton =  
    new JButton("SEARCH");  
  
styleButton(  
    searchButton,  
    BLUE  
);  
  
JPanel vehiclePanel =  
    new JPanel(  
        new BorderLayout(8, 0)  
    );  
  
vehiclePanel.setBackground(CARD);  
  
vehiclePanel.add(  
    vehicleField,  
    BorderLayout.CENTER  
);  
  
vehiclePanel.add(  
    searchButton,  
    BorderLayout.EAST  
);  
  
addRow(  
    content,  
    gbc,  
    row++,  
    vehicleLabel,  
    vehiclePanel  
);  
  
vehicleInfo =  
    createInfoLabel(  
        "Vehicle: -"
```

```

    );

    entryInfo =
        createInfoLabel(
            "Entry Time: -"
        );

    durationInfo =
        createInfoLabel(
            "Duration: -"
        );

    amountInfo =
        createInfoLabel(
            "Parking Amount: ₹0.00"
        );

    gbc.gridx = 0;
    gbc.gridy = row++;
    gbc.gridwidth = 2;

    content.add(
        vehicleInfo,
        gbc
    );

    gbc.gridy = row++;

    content.add(
        entryInfo,
        gbc
    );

    gbc.gridy = row++;

    content.add(
        durationInfo,
        gbc
    );

```



```

gbc.gridy = row++;

amountInfo.setForeground(GREEN);

amountInfo.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        18
    )
);

content.add(
    amountInfo,
    gbc
);

JLabel paymentLabel =
    createLabel(
        "Payment Method"
    );

paymentMethod =
    new JComboBox<>(
        new String[]{
            "Cash",
            "UPI",
            "Card"
        }
    );

styleComboBox(
    paymentMethod
);

gbc.gridwidth = 1;

addRow(
    content,
    gbc,

```

```

        row++,
        paymentLabel,
        paymentMethod
    );

    main.add(
        content,
        BorderLayout.CENTER
    );

    JPanel bottom =
        new JPanel(
            new FlowLayout(
                FlowLayout.RIGHT,
                10,
                10
            )
        );

    bottom.setBackground(ROOT);

    JButton cancel =
        new JButton("CANCEL");

    styleButton(
        cancel,
        BORDER
    );

    cancel.addActionListener(
        e -> dispose()
    );

    JButton checkout =
        new JButton(
            "✓ COMPLETE EXIT"
        );

    styleButton(
        checkout,

```

```

        GREEN
    );

    checkout.addActionListener(
        e -> completeExit()
    );

    searchButton.addActionListener(
        e -> searchVehicle()
    );

    vehicleField.addActionListener(
        e -> searchVehicle()
    );

    bottom.add(cancel);
    bottom.add(checkout);

    main.add(
        bottom,
        BorderLayout.SOUTH
    );

    setContentPane(main);
}

private JLabel createLabel(
    String text
) {

    JLabel label =
        new JLabel(text);

    label.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            13
        )
    );
}

```

```

        label.setForeground(TEXT);

        return label;
    }

    private JLabel createInfoLabel(
        String text
    ) {

        JLabel label =
            new JLabel(text);

        label.setFont(
            new Font(
                "Segoe UI",
                Font.PLAIN,
                14
            )
        );

        label.setForeground(MUTED);

        return label;
    }

    private JTextField createTextField() {

        JTextField field =
            new JTextField();

        field.setFont(
            new Font(
                "Segoe UI",
                Font.PLAIN,
                14
            )
        );

        field.setForeground(TEXT);
    }

```

```

field.setBackground(ROOT);

field.setCaretColor(BLUE);

field.setBorder(
    BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(
            BORDER
        ),
        BorderFactory.createEmptyBorder(
            8, 10, 8, 10
        )
    )
);

return field;
}

private void styleComboBox(
    JComboBox<String> combo
) {

    combo.setFont(
        new Font(
            "Segoe UI",
            Font.PLAIN,
            14
        )
    );

    combo.setForeground(TEXT);

    combo.setBackground(ROOT);

    combo.setBorder(
        BorderFactory.createLineBorder(
            BORDER
        )
    );
};

```

```

}

private void styleButton(
    JButton button,
    Color background
) {

    button.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            12
        )
    );

    button.setForeground(TEXT);

    button.setBackground(background);

    button.setFocusPainted(false);

    button.setBorder(
        BorderFactory.createEmptyBorder(
            10, 18, 10, 18
        )
    );

    button.setCursor(
        Cursor.getPredefinedCursor(
            Cursor.HAND_CURSOR
        )
    );
}

private void addRow(
    JPanel panel,
    GridBagConstraints gbc,
    int row,
    JLabel label,
    JComponent component

```

```

    ) {

        gbc.gridx = 0;
        gbc.gridy = row;
        gbc.gridwidth = 1;
        gbc.weightx = 0.3;

        panel.add(
            label,
            gbc
        );

        gbc.gridx = 1;
        gbc.weightx = 0.7;

        panel.add(
            component,
            gbc
        );
    }

    private void searchVehicle() {

        String vehicleNumber =
            vehicleField
                .getText()
                .trim()
                .toUpperCase();

        if (vehicleNumber.isEmpty()) {

            JOptionPane.showMessageDialog(
                this,
                "Enter a vehicle number.",
                "Input Required",
                JOptionPane.WARNING_MESSAGE
            );

            return;
        }
    }

```

```
String sql =
    "SELECT ps.session_id, " +
        "ps.entry_time, " +
        "ps.slot_id, " +
        "v.vehicle_number, " +
        "p.slot_number " +
        "FROM parking_sessions ps " +
        "JOIN vehicles v " +
        "ON ps.vehicle_id = v.vehicle_id " +
        "JOIN parking_slots p " +
        "ON ps.slot_id = p.slot_id " +
        "WHERE v.vehicle_number = ? " +
        "AND ps.session_status = 'Active' " +
        "LIMIT 1";
```

```
try (
    Connection con =
        DBConnection.getConnection();
```

```
    PreparedStatement ps =
        con.prepareStatement(sql)
) {
```

```
    ps.setString(
        1,
        vehicleNumber
    );
```

```
    try (
        ResultSet rs =
            ps.executeQuery()
    ) {
```

```
        if (rs.next()) {

            int sessionId =
                rs.getInt(
                    "session_id"
                );
```



```
Timestamp timestamp =
    rs.getTimestamp(
        "entry_time"
    );

int slotId =
    rs.getInt(
        "slot_id"
    );

String number =
    rs.getString(
        "vehicle_number"
    );

String slotNumber =
    rs.getString(
        "slot_number"
    );

LocalDateTime entry =
    timestamp
        .toLocalDateTime();

LocalDateTime now =
    LocalDateTime.now();

long minutes =
    Duration
        .between(
            entry,
            now
        )
        .toMinutes();

double fee =
    sessionDAO.calculateFee(
        minutes
    );
```

```
vehicleInfo.setText(  
    "Vehicle: "  
        + number  
        + " | Slot: "  
        + slotNumber  
);
```

```
entryInfo.setText(  
    "Entry Time: "  
        + entry  
);
```

```
durationInfo.setText(  
    "Duration: "  
        + formatDuration(  
            minutes  
        )  
);
```

```
amountInfo.setText(  
    String.format(  
        "Parking Amount: ₹%.2f",  
        fee  
    )  
);
```

```
vehicleInfo.putClientProperty(  
    "sessionId",  
    sessionId  
);
```

```
vehicleInfo.putClientProperty(  
    "slotId",  
    slotId  
);
```

```
vehicleInfo.putClientProperty(  
    "entryTime",  
    entry  
);
```

```

    );

    vehicleInfo.putClientProperty(
        "fee",
        fee
    );

} else {

    clearInformation();

    JOptionPane.showMessageDialog(
        this,
        "No active parking session found for "
        + vehicleNumber,
        "Vehicle Not Found",
        JOptionPane.WARNING_MESSAGE
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database error:\n"
        + e.getMessage(),
        "Database Error",
        JOptionPane.ERROR_MESSAGE
    );
}

}

private String formatDuration(
    long minutes
) {

    long hours =
        minutes / 60;

```

```

long remaining =
    minutes % 60;

if (hours == 0) {

    return remaining
        + " minutes";
}

return hours
    + " hour(s) "
    + remaining
    + " minute(s)";
}

private void completeExit() {

    Object sessionObject =
        vehicleInfo.getClientProperty(
            "sessionId"
        );

    Object slotObject =
        vehicleInfo.getClientProperty(
            "slotId"
        );

    Object feeObject =
        vehicleInfo.getClientProperty(
            "fee"
        );

    if (sessionObject == null ||
        slotObject == null ||
        feeObject == null) {

        JOptionPane.showMessageDialog(
            this,
            "Search an active vehicle first.",
            "Vehicle Required",

```

```

        JOptionPane.WARNING_MESSAGE
    );

    return;
}

int sessionId =
    (Integer) sessionObject;

int slotId =
    (Integer) slotObject;

double fee =
    (Double) feeObject;

String method =
    String.valueOf(
        paymentMethod.getSelectedItemAt()
    );

int confirm =
    JOptionPane.showConfirmDialog(
        this,
        "Parking Amount: ₹"
            + String.format(
                "%.2f",
                fee
            )
            + "\nPayment Method: "
            + method
            + "\n\n"
            + "Complete vehicle exit?",
        "Confirm Exit",
        JOptionPane.YES_NO_OPTION
    );

if (confirm !=
    JOptionPane.YES_OPTION) {

    return;
}

```

```
}
```

```
Connection con = null;
```

```
try {
```

```
    con =
```

```
        DBConnection.getConnection();
```

```
    con.setAutoCommit(false);
```

```
    String updateSession =
```

```
        "UPDATE parking_sessions " +
```

```
        "SET exit_time = NOW(), " +
```

```
        "duration_minutes = TIMESTAMPDIFF(" +
```

```
        "MINUTE, entry_time, NOW()), " +
```

```
        "parking_amount = ?, " +
```

```
        "session_status = 'Completed' " +
```

```
        "WHERE session_id = ?";
```

```
    try (
```

```
        PreparedStatement ps =
```

```
            con.prepareStatement(
```

```
                updateSession
```

```
            )
```

```
    ) {
```

```
        ps.setDouble(
```

```
            1,
```

```
            fee
```

```
        );
```

```
        ps.setInt(
```

```
            2,
```

```
            sessionId
```

```
        );
```

```
        ps.executeUpdate();
```

```
    }
```

```

String updateSlot =
    "UPDATE parking_slots " +
        "SET slot_status = 'Available' " +
        "WHERE slot_id = ?";

try (
    PreparedStatement ps =
        con.prepareStatement(
            updateSlot
        )
) {

    ps.setInt(
        1,
        slotId
    );

    ps.executeUpdate();
}

String paymentSql =
    "INSERT INTO payments " +
        "(session_id, amount, payment_method, " +
        "payment_status) " +
        "VALUES (?, ?, ?, 'Paid')";

try (
    PreparedStatement ps =
        con.prepareStatement(
            paymentSql
        )
) {

    ps.setInt(
        1,
        sessionId
    );

    ps.setDouble(
        2,

```

```

        fee
    );

    ps.setString(
        3,
        method
    );

    ps.executeUpdate();
}

con.commit();

JOptionPane.showMessageDialog(
    this,
    "VEHICLE EXIT SUCCESSFUL\n\n"
        + "Session ID: "
        + sessionId
        + "\n"
        + "Amount Paid: ₹"
        + String.format(
            "%.2f",
            fee
        )
        + "\n"
        + "Payment: "
        + method
        + "\n"
        + "Slot Status: AVAILABLE",
    "Payment Successful",
    JOptionPane.INFORMATION_MESSAGE
);

dispose();

} catch (SQLException e) {

    try {

        if (con != null) {

```



```

        con.rollback();
    }

} catch (SQLException ignored) {
}

JOptionPane.showMessageDialog(
    this,
    "Exit processing failed:\n"
        + e.getMessage(),
    "Database Error",
    JOptionPane.ERROR_MESSAGE
);

} finally {

    try {

        if (con != null) {
            con.setAutoCommit(true);
            con.close();
        }

    } catch (SQLException ignored) {
    }

}

}

private void clearInformation() {

    vehicleInfo.setText(
        "Vehicle: -"
    );

    entryInfo.setText(
        "Entry Time: -"
    );

    durationInfo.setText(
        "Duration: -"
    );

```

```

    );

    amountInfo.setText(
        "Parking Amount: ₹0.00"
    );

    vehicleInfo.putClientProperty(
        "sessionId",
        null
    );

    vehicleInfo.putClientProperty(
        "slotId",
        null
    );

    vehicleInfo.putClientProperty(
        "entryTime",
        null
    );

    vehicleInfo.putClientProperty(
        "fee",
        null
    );
}
}

package com.campus.parking.ui;
import com.campus.parking.ui.dialogs.CheckInDialog;
import javax.swing.*;
import java.awt.*;

public class GateConsolePanel extends JPanel {

    public GateConsolePanel() {

        setLayout(new GridLayout(1, 2, 20, 20));
        setBackground(new Color(13, 22, 42));
        setBorder(BorderFactory.createEmptyBorder(35, 35, 35, 35));
    }
}

```

```

    add(createEntryPanel());
    add(createExitPanel());
}

private JPanel createEntryPanel() {

    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setBackground(new Color(30, 43, 65));
    panel.setBorder(
        BorderFactory.createEmptyBorder(40, 40, 40, 40)
    );

    JLabel title = new JLabel("VEHICLE ENTRY");
    title.setFont(new Font("Segoe UI", Font.BOLD, 26));
    title.setForeground(Color.WHITE);
    title.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel info = new JLabel(
        "<html><center>Register a vehicle and assign<br>" +
        "an available parking slot.</center></html>"
    );
    info.setFont(new Font("Segoe UI", Font.PLAIN, 15));
    info.setForeground(new Color(160, 180, 215));
    info.setAlignmentX(Component.CENTER_ALIGNMENT);

    JButton button = new JButton("CHECK IN VEHICLE");
    button.setAlignmentX(Component.CENTER_ALIGNMENT);

    button.addActionListener(e -> openCheckIn());

    panel.add(Box.createVerticalGlue());
    panel.add(title);
    panel.add(Box.createVerticalStrut(20));
    panel.add(info);
    panel.add(Box.createVerticalStrut(40));
    panel.add(button);
    panel.add(Box.createVerticalGlue());

    return panel;
}

```

```
}
```

```
private JPanel createExitPanel() {

    JPanel panel = new JPanel();
    panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
    panel.setBackground(new Color(30, 43, 65));
    panel.setBorder(
        BorderFactory.createEmptyBorder(40, 40, 40, 40)
    );

    JLabel title = new JLabel("VEHICLE EXIT");
    title.setFont(new Font("Segoe UI", Font.BOLD, 26));
    title.setForeground(Color.WHITE);
    title.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel info = new JLabel(
        "<html><center>Calculate parking duration,<br>" +
        "fee and complete payment.</center></html>"
    );
    info.setFont(new Font("Segoe UI", Font.PLAIN, 15));
    info.setForeground(new Color(160, 180, 215));
    info.setAlignmentX(Component.CENTER_ALIGNMENT);

    JButton button = new JButton("PROCESS VEHICLE EXIT");
    button.setAlignmentX(Component.CENTER_ALIGNMENT);

    button.addActionListener(e -> processExit());

    panel.add(Box.createVerticalGlue());
    panel.add(title);
    panel.add(Box.createVerticalStrut(20));
    panel.add(info);
    panel.add(Box.createVerticalStrut(40));
    panel.add(button);
    panel.add(Box.createVerticalGlue());

    return panel;
}
```

```

private void openCheckIn() {

    Window window =
        SwingUtilities.getWindowAncestor(this);

    if (window instanceof Frame) {

        CheckInDialog dialog =
            new CheckInDialog((Frame) window);

        dialog.setLocationRelativeTo(window);
        dialog.setVisible(true);
    }
}

private void processExit() {

    JOptionPane.showMessageDialog(
        this,
        "Vehicle Exit & Payment module is ready.",
        "Vehicle Exit",
        JOptionPane.INFORMATION_MESSAGE
    );
}

public void refreshData() {
    repaint();
}
}

package com.campus.parking.ui;

import com.campus.parking.ui.dialogs.CheckInDialog;
import com.campus.parking.ui.panels.DashboardPanel;
import com.campus.parking.ui.panels.PaymentsPanel;
import com.campus.parking.ui.panels.SlotMatrixPanel;
import com.campus.parking.ui.panels.UserManagementPanel;
import com.campus.parking.ui.panels.ViolationsPanel;

import javax.swing.*;
import java.awt.*;

```

```
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class MainAppFrame extends JFrame {

    // =====
    // COLORS
    // =====

    private static final Color BACKGROUND =
        new Color(10, 18, 35);

    private static final Color TOP_BAR =
        new Color(8, 15, 30);

    private static final Color SIDEBAR =
        new Color(28, 42, 64);

    private static final Color PANEL =
        new Color(29, 43, 65);

    private static final Color PANEL_DARK =
        new Color(17, 28, 48);

    private static final Color FIELD =
        new Color(10, 18, 35);

    private static final Color BORDER =
        new Color(55, 75, 105);

    private static final Color TEXT =
        new Color(245, 247, 250);

    private static final Color MUTED =
        new Color(145, 170, 205);

    private static final Color BLUE =
        new Color(38, 175, 240);
```

```

private static final Color GREEN =
    new Color(12, 195, 140);

private static final Color RED =
    new Color(245, 70, 70);

private static final Color ORANGE =
    new Color(255, 175, 0);

private static final Color PURPLE =
    new Color(92, 82, 235);

private JPanel contentPanel;
private JLabel timeLabel;

// =====
// CONSTRUCTOR
// =====

public MainAppFrame() {

    setTitle("Smart Campus Parking & Traffic Sentinel");

    setSize(1650, 900);

    setMinimumSize(
        new Dimension(1100, 700)
    );

    setLocationRelativeTo(null);

    setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE
    );

    getContentPane().setBackground(
        BACKGROUND
    );

```

```

        setLayout(
            new BorderLayout()
        );

        createTopBar();

        createSidebar();

        contentPanel =
            new JPanel(
                new BorderLayout()
            );

        contentPanel.setBackground(
            BACKGROUND
        );

        add(
            contentPanel,
            BorderLayout.CENTER
        );

        showDashboard();

        startClock();
    }

// =====
// TOP BAR
// =====

private void createTopBar() {

    JPanel topBar =
        new JPanel(
            new BorderLayout()
        );

    topBar.setPreferredSize(
        new Dimension(0, 82)
    );

```



```

);

topBar.setBackground(
    TOP_BAR
);

topBar.setBorder(
    BorderFactory.createMatteBorder(
        0,
        0,
        1,
        0,
        BORDER
    )
);

// SEARCH AREA

JPanel left =
    new JPanel(
        new FlowLayout(
            FlowLayout.LEFT,
            20,
            15
        )
    );

left.setOpaque(false);

JTextField searchField =
    new JTextField();

searchField.setText(
    "Search vehicle, slot or session..."
);

searchField.setPreferredSize(
    new Dimension(540, 52)
);

```

```

searchField.setFont(
    new Font(
        "Segoe UI",
        Font.PLAIN,
        15
    )
);

searchField.setForeground(
    MUTED
);

searchField.setBackground(
    FIELD
);

searchField.setCaretColor(
    Color.WHITE
);

searchField.setBorder(
    BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(
            BORDER
        ),
        BorderFactory.createEmptyBorder(
            0,
            12,
            0,
            12
        )
    )
);

left.add(searchField);

// RIGHT AREA

JPanel right =
    new JPanel(

```

```

        new FlowLayout(
            FlowLayout.RIGHT,
            12,
            15
        )
    );

    right.setOpaque(false);

    timeLabel =
        new JLabel();

    timeLabel.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            16
        )
    );

    timeLabel.setForeground(
        TEXT
    );

    right.add(timeLabel);

    JLabel live =
        new JLabel("● LIVE");

    live.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            14
        )
    );

    live.setForeground(
        GREEN
    );

```

```

right.add(live);

JButton checkInButton =
    createButton(
        "+ VEHICLE CHECK-IN",
        PURPLE
    );

checkInButton.setPreferredSize(
    new Dimension(
        225,
        46
    )
);

checkInButton.addActionListener(
    e -> openCheckIn()
);

right.add(checkInButton);

topBar.add(
    left,
    BorderLayout.WEST
);

topBar.add(
    right,
    BorderLayout.EAST
);

add(
    topBar,
    BorderLayout.NORTH
);
}

// =====
// SIDEBAR

```

```
// =====
```

```
private void createSidebar() {
```

```
    JPanel sidebar =  
        new JPanel(  
            new BorderLayout()  
        );
```

```
    sidebar.setPreferredSize(  
        new Dimension(  
            340,  
            0  
        )  
    );
```

```
    sidebar.setBackground(  
        SIDEBAR  
    );
```

```
// LOGO
```

```
    JPanel logoPanel =  
        new JPanel();
```

```
    logoPanel.setLayout(  
        new BoxLayout(  
            logoPanel,  
            BoxLayout.Y_AXIS  
        )  
    );
```

```
    logoPanel.setBackground(  
        SIDEBAR  
    );
```

```
    logoPanel.setBorder(  
        BorderFactory.createEmptyBorder(  
            40,  
            22,
```

```
        20,  
        22  
    )  
);  
  
JLabel logo =  
    new JLabel("▣");  
  
logo.setFont(  
    new Font(  
        "Segoe UI",  
        Font.BOLD,  
        30  
    )  
);  
  
logo.setForeground(  
    BLUE  
);  
  
JLabel title =  
    new JLabel(  
        "SMART CAMPUS"  
    );  
  
title.setFont(  
    new Font(  
        "Segoe UI",  
        Font.BOLD,  
        26  
    )  
);  
  
title.setForeground(  
    TEXT  
);  
  
JLabel subtitle =  
    new JLabel(  
        "PARKING SENTINEL"
```

```
    );

    subtitle.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            14
        )
    );

    subtitle.setForeground(
        MUTED
    );

    logoPanel.add(logo);

    logoPanel.add(
        Box.createVerticalStrut(20)
    );

    logoPanel.add(title);

    logoPanel.add(
        Box.createVerticalStrut(4)
    );

    logoPanel.add(subtitle);

    sidebar.add(
        logoPanel,
        BorderLayout.NORTH
    );

    // MENU

    JPanel menu =
        new JPanel();

    menu.setLayout(
        new BoxLayout(
```

```

        menu,
        BoxLayout.Y_AXIS
    )
);

menu.setBackground(
    SIDEBAR
);

menu.setBorder(
    BorderFactory.createEmptyBorder(
        10,
        16,
        10,
        16
    )
);

JButton dashboard =
    createMenuButton(
        "▣ Command Dashboard"
    );

JButton slots =
    createMenuButton(
        "▣ Live Slot Matrix"
    );

JButton gate =
    createMenuButton(
        "▣ ENTRY / EXIT GATE CONSOLE"
    );

JButton users =
    createMenuButton(
        "▣ User Directory"
    );

JButton violations =
    createMenuButton(

```



```
        "☐ Violations"
    );

    JButton payments =
        createMenuButton(
            "₹ Payments"
        );

    dashboard.addActionListener(
        e -> showDashboard()
    );

    slots.addActionListener(
        e -> showSlots()
    );

    gate.addActionListener(
        e -> showGate()
    );

    users.addActionListener(
        e -> showUsers()
    );

    violations.addActionListener(
        e -> showViolations()
    );

    payments.addActionListener(
        e -> showPayments()
    );

    menu.add(dashboard);

    menu.add(
        Box.createVerticalStrut(8)
    );

    menu.add(slots);
```

```
menu.add(
    Box.createVerticalStrut(8)
);

menu.add(gate);

menu.add(
    Box.createVerticalStrut(8)
);

menu.add(users);

menu.add(
    Box.createVerticalStrut(8)
);

menu.add(violations);

menu.add(
    Box.createVerticalStrut(8)
);

menu.add(payments);

sidebar.add(
    menu,
    BorderLayout.CENTER
);

// SYSTEM STATUS

JPanel status =
    new JPanel(
        new BorderLayout()
    );

status.setPreferredSize(
    new Dimension(
        0,
        95
    )
);
```

```

    )
);

status.setBackground(
    SIDEBAR
);

status.setBorder(
    BorderFactory.createMatteBorder(
        1,
        0,
        0,
        0,
        BORDER
    )
);

JPanel statusContent =
    new JPanel();

statusContent.setLayout(
    new BoxLayout(
        statusContent,
        BoxLayout.Y_AXIS
    )
);

statusContent.setOpaque(false);

statusContent.setBorder(
    BorderFactory.createEmptyBorder(
        12,
        20,
        10,
        10
    )
);

JLabel statusTitle =
    new JLabel(

```

```
        "SYSTEM STATUS"
    );

statusTitle.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        12
    )
);

statusTitle.setForeground(
    MUTED
);

JLabel database =
    new JLabel(
        "● Database Connected"
    );

database.setFont(
    new Font(
        "Segoe UI",
        Font.PLAIN,
        13
    )
);

database.setForeground(
    GREEN
);

JLabel engine =
    new JLabel(
        "● Sentinel Engine Online"
    );

engine.setFont(
    new Font(
        "Segoe UI",
```

```
        Font.PLAIN,
        13
    )
);

engine.setForeground(
    GREEN
);

statusContent.add(
    statusTitle
);

statusContent.add(
    Box.createVerticalStrut(8)
);

statusContent.add(
    database
);

statusContent.add(
    Box.createVerticalStrut(5)
);

statusContent.add(
    engine
);

status.add(
    statusContent,
    BorderLayout.CENTER
);

sidebar.add(
    status,
    BorderLayout.SOUTH
);

add(
```

```

        sidebar,
        BorderLayout.WEST
    );
}

// =====
// MENU BUTTON
// =====

private JButton createMenuButton(
    String text
) {

    JButton button =
        new JButton(text);

    button.setAlignmentX(
        Component.LEFT_ALIGNMENT
    );

    button.setMaximumSize(
        new Dimension(
            Integer.MAX_VALUE,
            60
        )
    );

    button.setPreferredSize(
        new Dimension(
            300,
            60
        )
    );

    button.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            16
        )
    );
}

```

```
);

button.setForeground(
    TEXT
);

button.setBackground(
    SIDEBAR
);

button.setHorizontalAlignment(
    SwingConstants.LEFT
);

button.setBorder(
    BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(
            SIDEBAR
        ),
        BorderFactory.createEmptyBorder(
            0,
            15,
            0,
            10
        )
    )
);

button.setFocusPainted(false);

button.setFocusable(false);

button.setOpaque(true);

button.setContentAreaFilled(true);

button.setCursor(
    new Cursor(
        Cursor.HAND_CURSOR
    )
);
```

```
);
```

```
button.addMouseListener(  
    new MouseAdapter() {
```

```
        @Override  
        public void mouseEntered(  
            MouseEvent e  
        ) {
```

```
            button.setBackground(  
                new Color(  
                    43,  
                    62,  
                    88  
                )  
            );
```

```
            button.setBorder(  
                BorderFactory.createCompoundBorder(  
                    BorderFactory.createLineBorder(  
                        BLUE  
                    ),  
                    BorderFactory.createEmptyBorder(  
                        0,  
                        15,  
                        0,  
                        10  
                    )  
                )  
            );  
        }  
    }
```

```
        @Override  
        public void mouseExited(  
            MouseEvent e  
        ) {
```

```
            button.setBackground(  
                SIDEBAR
```



```

    );

    button.setBorder(
        BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(
                SIDEBAR
            ),
            BorderFactory.createEmptyBorder(
                0,
                15,
                0,
                10
            )
        )
    );
}
}
);

return button;
}

```

```

// =====
// NORMAL BUTTON
// =====

```

```

private JButton createButton(
    String text,
    Color color
) {

```

```

    JButton button =
        new JButton(text);

```

```

    button.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            14
        )
    )

```

```

    );

    button.setForeground(
        Color.WHITE
    );

    button.setBackground(
        color
    );

    button.setFocusPainted(false);

    button.setFocusable(false);

    button.setBorderPainted(false);

    button.setOpaque(true);

    button.setContentAreaFilled(true);

    button.setCursor(
        new Cursor(
            Cursor.HAND_CURSOR
        )
    );

    return button;
}

// =====
// SHOW PANEL
// =====

private void showPanel(
    JPanel panel
) {

    contentPanel.removeAll();

    contentPanel.add(

```

```

        panel,
        BorderLayout.CENTER
    );

    contentPanel.revalidate();

    contentPanel.repaint();
}

// =====
// DASHBOARD
// =====

private void showDashboard() {

    try {

        DashboardPanel panel =
            new DashboardPanel();

        showPanel(panel);

    } catch (Exception e) {

        showError(
            "Unable to open Dashboard.",
            e
        );
    }
}

// =====
// SLOT MATRIX
// =====

private void showSlots() {

    try {

        SlotMatrixPanel panel =

```

```

        new SlotMatrixPanel();

        showPanel(panel);

    } catch (Exception e) {

        showError(
            "Unable to open Slot Matrix.",
            e
        );
    }
}

// =====
// GATE CONSOLE
// =====

private void showGate() {

    try {

        GateConsolePanel panel =
            new GateConsolePanel();

        showPanel(panel);

    } catch (Exception e) {

        showError(
            "Unable to open Gate Console.",
            e
        );
    }
}

// =====
// USERS
// =====

private void showUsers() {

```

```

try {

    UserManagementPanel panel =
        new UserManagementPanel();

    showPanel(panel);

} catch (Exception e) {

    showError(
        "Unable to open User Directory.",
        e
    );
}
}

```

```

// =====
// VIOLATIONS
// =====

```

```

private void showViolations() {

    try {

        ViolationsPanel panel =
            new ViolationsPanel();

        showPanel(panel);

    } catch (Exception e) {

        showError(
            "Unable to open Violations.",
            e
        );
    }
}

```

```

// =====

```

```

// PAYMENTS
// =====

private void showPayments() {

    try {

        PaymentsPanel panel =
            new PaymentsPanel();

        showPanel(panel);

    } catch (Exception e) {

        showError(
            "Unable to open Payments.",
            e
        );
    }
}

// =====
// CHECK-IN
// =====

private void openCheckIn() {

    try {

        CheckInDialog dialog =
            new CheckInDialog(this);

        dialog.setLocationRelativeTo(this);

        dialog.setVisible(true);

    } catch (Exception e) {

        showError(
            "Unable to open Vehicle Check-In.",

```

```

        e
    );
}
}

// =====
// CLOCK
// =====

private void startClock() {

    Timer timer =
        new Timer(
            1000,
            e -> updateClock()
        );

    timer.start();

    updateClock();
}

private void updateClock() {

    if (timeLabel != null) {

        String time =
            LocalDateTime.now()
                .format(
                    DateTimeFormatter.ofPattern(
                        "HH:mm:ss"
                    )
                );

        timeLabel.setText(
            time
        );
    }
}

```

```

// =====
// ERROR MESSAGE
// =====

private void showError(
    String message,
    Exception e
) {

    JOptionPane.showMessageDialog(
        this,
        message + "\n\n"
            + e.getMessage(),
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
}

// =====
// MAIN
// =====

public static void main(
    String[] args
) {

    SwingUtilities.invokeLater(
        () -> {

            /*
             * IMPORTANT:
             * Do NOT use Windows/System
             * LookAndFeel here.
             *
             * We use Metal LookAndFeel
             * so Swing does not turn
             * our dark buttons white.
             */

            try {

```



```
        UIManager.setLookAndFeel(
            UIManager.getCrossPlatformLookAndFeelClassName()
        );

    } catch (Exception ignored) {
    }

    // Global Swing colors

    UIManager.put(
        "Button.background",
        SIDEBAR
    );

    UIManager.put(
        "Button.foreground",
        TEXT
    );

    UIManager.put(
        "Button.select",
        BLUE
    );

    UIManager.put(
        "TextField.background",
        FIELD
    );

    UIManager.put(
        "TextField.foreground",
        TEXT
    );

    UIManager.put(
        "TextField.caretForeground",
        Color.WHITE
    );
```

```
UIManager.put(  
    "ComboBox.background",  
    FIELD  
);
```

```
UIManager.put(  
    "ComboBox.foreground",  
    TEXT  
);
```

```
UIManager.put(  
    "Table.background",  
    PANEL  
);
```

```
UIManager.put(  
    "Table.foreground",  
    TEXT  
);
```

```
UIManager.put(  
    "Table.selectionBackground",  
    new Color(  
        45,  
        90,  
        135  
    )  
);
```

```
UIManager.put(  
    "Table.selectionForeground",  
    Color.WHITE  
);
```

```
UIManager.put(  
    "TableHeader.background",  
    FIELD  
);
```

```
UIManager.put(  
    "TableHeader.foreground",  
    TEXT  
);
```

```

        "TableHeader.foreground",
        TEXT
    );

    UIManager.put(
        "Panel.background",
        BACKGROUND
    );

    MainAppFrame frame =
        new MainAppFrame();

    frame.setVisible(true);
}
);
}
}

package com.campus.parking;

import com.campus.parking.database.DatabaseBootstrap;
import com.campus.parking.ui.MainAppFrame;

import javax.swing.*;

public class Main {

    public static void main(String[] args) {

        DatabaseBootstrap.initialize();

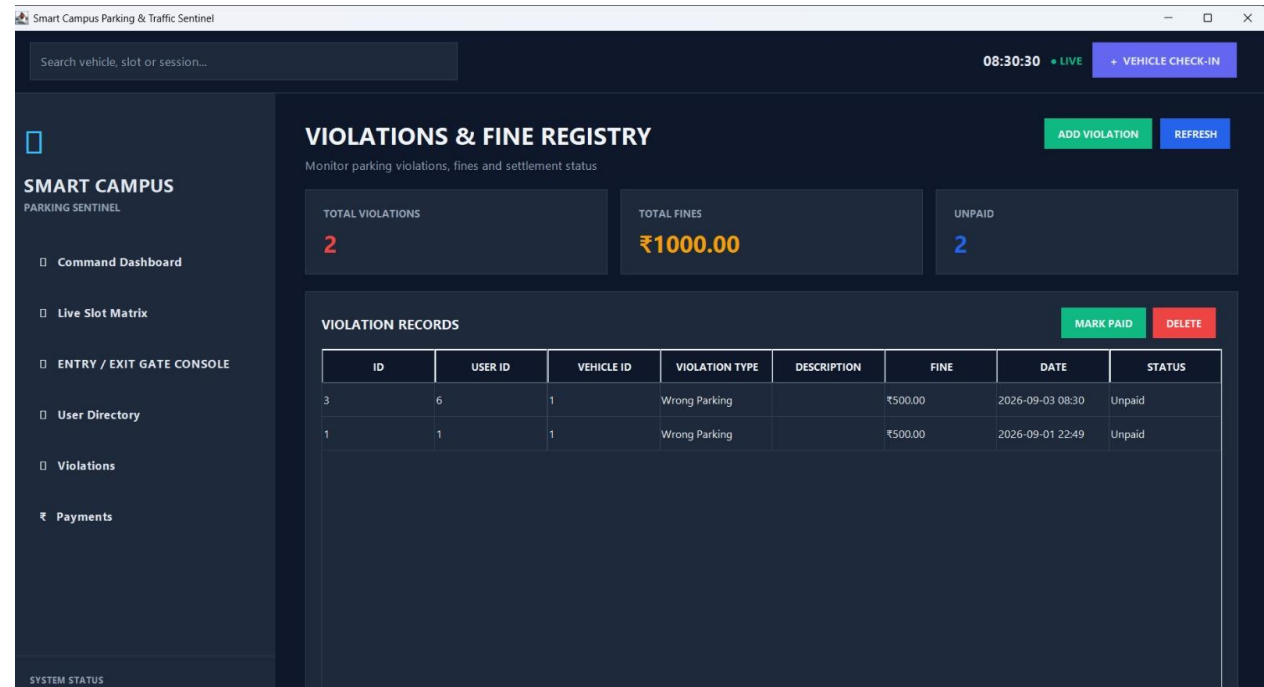
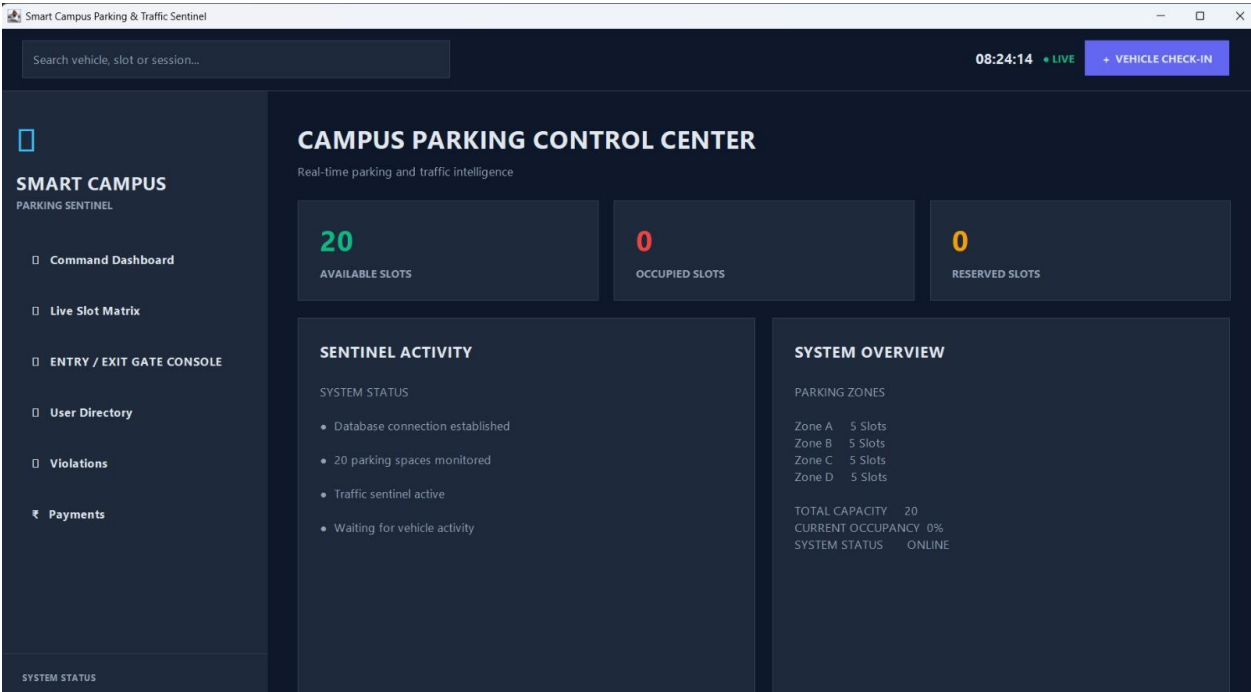
        SwingUtilities.invokeLater(() -> {

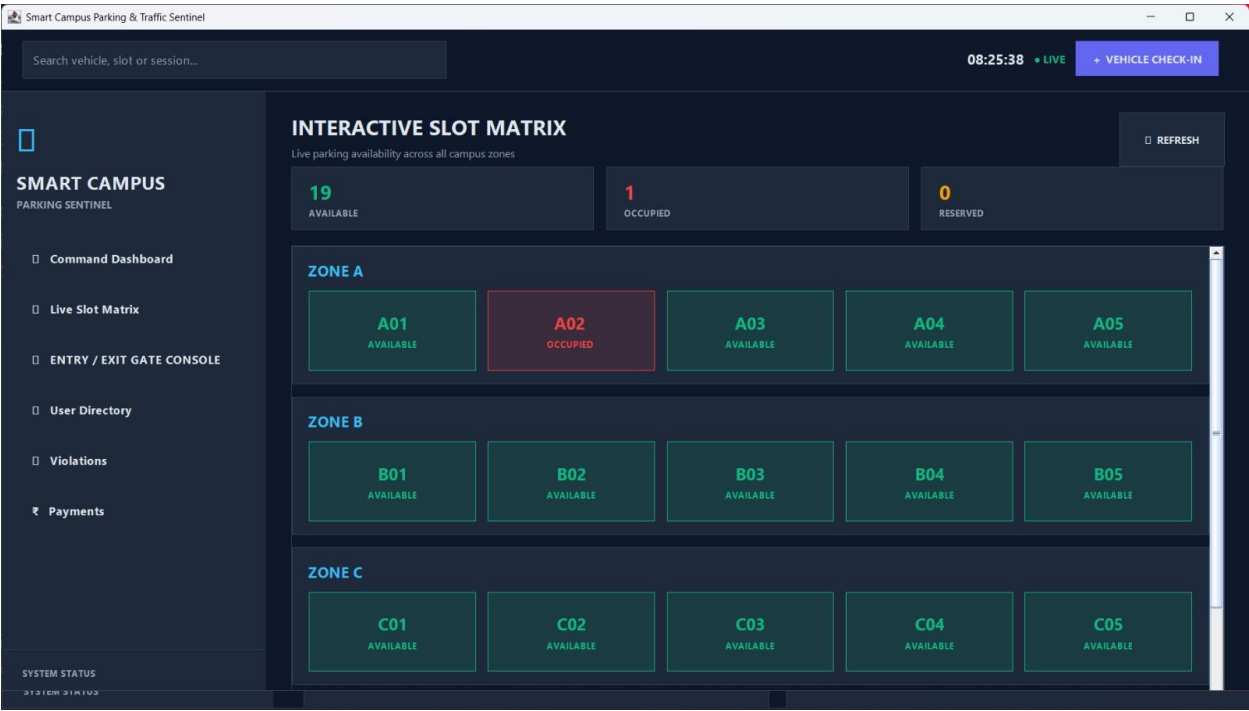
            MainAppFrame frame =
                new MainAppFrame();

            frame.setVisible(true);
        });
    }
}

```

Screenshots:





Smart Campus Parking & Traffic Sentinel

Search vehicle, slot or session...

08:28:03 • LIVE + VEHICLE CHECK-IN

USER & VEHICLE DIRECTORY

Manage campus users, visitors and registered vehicles

SEARCH REFRESH + ADD USER

TOTAL USERS **5**

STUDENTS **2**

STAFF **1**

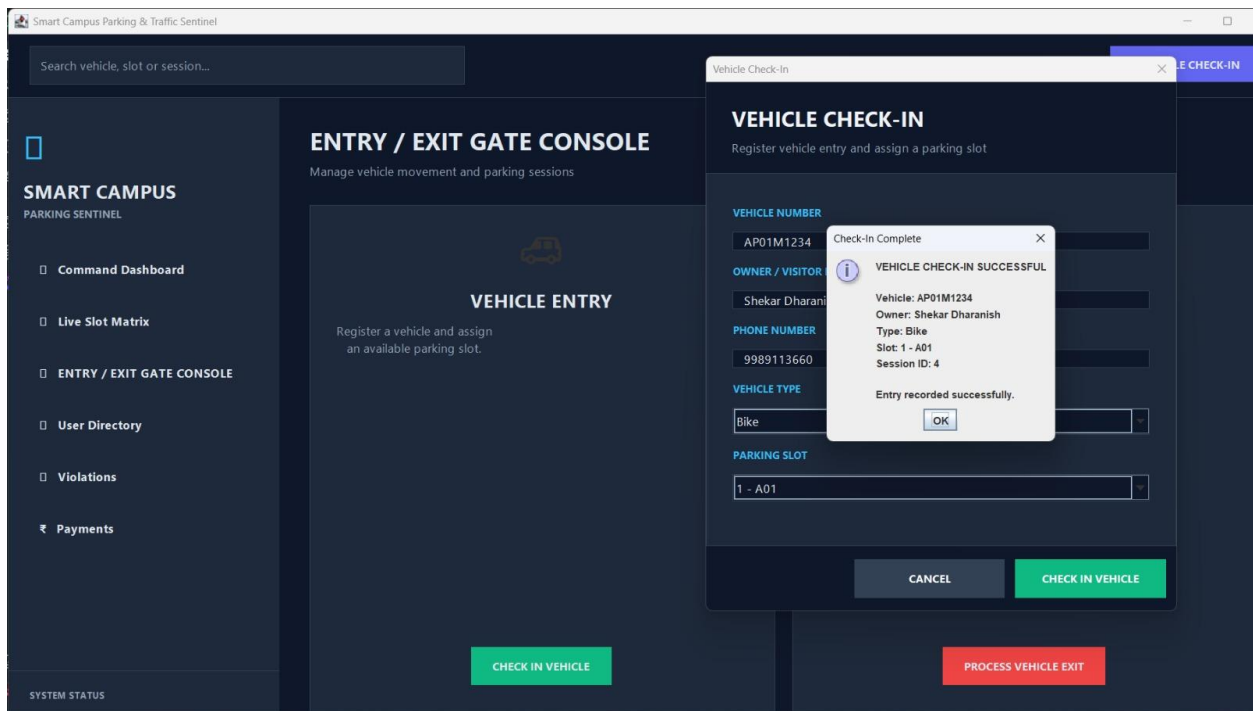
REGISTERED CAMPUS USERS

ID	NAME	EMAIL	PHONE	USER TYPE	ACTION
6	Shekar Dharanish	9989113660@campus.local	9989113660	Student	DOUBLE CLICK TO EDIT
5	Dharanish	9391143609@campus.local	9391143609	Student	DOUBLE CLICK TO EDIT
4	Kavin Kumar	kavin@gmail.com	9876543213	Visitor	DOUBLE CLICK TO EDIT
3	Rahul Raj	rahul@gmail.com	9876543212	Staff	DOUBLE CLICK TO EDIT
2	Priya Sharma	priya@gmail.com	9876543211	Faculty	DOUBLE CLICK TO EDIT
1	Arjun Kumar	arjun@gmail.com	9876543210	Student	DOUBLE CLICK TO EDIT

SMART CAMPUS PARKING SENTINEL

- Command Dashboard
- Live Slot Matrix
- ENTRY / EXIT GATE CONSOLE
- User Directory
- Violations
- Payments

SYSTEM STATUS



Student Name: S. Dharanish

Register Number: 192472152

Project Title: Smart Campus Parking and Traffic Management System

Individual Role/Contribution

I worked as a frontend and GUI developer on the project. I was responsible for designing and implementing the graphical user interface of the Smart Campus Parking & Traffic Management System. I worked on creating user-friendly screens for monitoring parking slots, vehicle entry and exit, user management, violations, and payments.

Modules/Tasks Completed

- Designed and implemented the main application dashboard using Java Swing.
- Developed the Live Slot Matrix interface for displaying parking slot availability.
- Implemented the Entry / Exit Gate Console interface.
- Developed the Vehicle Check-In interface.
- Designed the User and Vehicle Directory screen.
- Implemented the Violations management interface.
- Designed the Payments management interface.
- Added buttons, forms, tables, dialogs, and navigation components.

Brief Description of Work

I contributed mainly to the frontend and user interface development of the Smart Campus Parking & Traffic Management System. I used Java Swing to create the application's dashboard and different management screens. The interface provides information about available, occupied, and reserved parking slots.

I also developed interfaces for vehicle check-in, vehicle exit, user management, violations, and payments. The GUI was integrated with the backend components so that the information entered by the user could be processed and displayed properly.

Outcome/Learning

Through this project, I gained practical experience in Java Swing GUI development, event handling, form design, tables, dialogs, user interaction, and frontend-backend integration. I also improved my understanding of designing user-friendly desktop applications using Java.

Student Name: P. Jeevan Kumar

Register Number: 192472082

Project Title: Smart Campus Parking and Traffic Management System

Individual Role/Contribution

I worked as a backend developer on the project. I was responsible for implementing the core application logic related to vehicle entry, parking slot assignment, vehicle exit, parking sessions, reservations, and payment processing.

Modules/Tasks Completed

- Developed the Java backend classes for parking operations.
- Implemented vehicle check-in functionality.
- Implemented automatic parking slot assignment.
- Developed vehicle exit processing functionality.
- Implemented parking session management.
- Developed parking reservation functionality.
- Implemented parking fee calculation.
- Developed payment processing logic.
- Implemented validation for vehicle and parking information.
- Handled different vehicle types and parking conditions.

Brief Description of Work

I contributed mainly to the backend functionality of the Smart Campus Parking & Traffic Management System. I developed Java classes responsible for handling the main application logic and processing parking-related operations.

I worked on vehicle check-in, parking slot allocation, parking sessions, vehicle exit, reservations, and payment processing. The backend receives information from the GUI, processes the required operations, and communicates with the database to store and retrieve information.

Outcome/Learning

Through this project, I gained practical experience in Java backend development, object-oriented programming, event handling, application logic, validation, parking management, fee calculation, and integration with GUI and database components. I also developed a better understanding of how different modules work together in a complete Java application.

Student Name: Akileshwaran A

Register Number: 192511354

Project Title: Smart Campus Parking and Traffic Management System

Individual Role/Contribution

I worked as a database and backend integration developer on the project. I was responsible for designing the MySQL database, developing JDBC connectivity, implementing DAO classes, and managing database operations required by the Smart Campus Parking & Traffic Management System.

Modules/Tasks Completed

- Designed and implemented the MySQL database structure.
- Created tables for users, vehicles, parking slots, sessions, reservations, violations, and payments.
- Developed JDBC database connectivity.
- Developed DAO classes for database operations.
- Implemented CRUD operations for required entities.
- Implemented user and vehicle data management.
- Worked on parking slot database operations.

Brief Description of Work

I contributed mainly to the database and backend integration functionality of the Smart Campus Parking & Traffic Management System. I designed the MySQL database and developed the required tables for storing users, vehicles, parking slots, parking sessions, reservations, violations, and payment information.

I also developed JDBC connectivity and DAO classes responsible for communicating with the database and performing operations such as inserting, retrieving, updating, and deleting records. The database components were integrated with the Java backend to provide reliable data storage and retrieval.

Outcome/Learning

Through this project, I gained practical experience in Java backend development, MySQL, JDBC, DAO architecture, database design, CRUD functionality, database integration, testing, and debugging. I also developed a better understanding of how Java applications communicate with and manage data in a relational database.